

Unidad 2

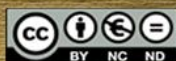


Centro Don Bosco
Villamuriel de Cerrato

Introducción a PHP

Apuntes realizados para la asignatura de FP Grado Superior:
Implantación de Aplicaciones Web
del ciclo Administración de Sistemas Informáticos en Red

Autor: Jorge Sánchez Asenjo (www.jorgesanchez.net)
Versión del documento: 2.01, Año 2012



Esta obra está bajo una licencia de Reconocimiento-NoComercial-CompartirIgual de Creative Commons.
Para ver una copia de esta licencia, visite: <http://creativecommons.org/licenses/by-nc-sa/3.0/deed.es>



Reconocimiento-NoComercial-CompartirIgual 2.5 España

Usted es libre de:



copiar, distribuir y comunicar públicamente la obra



hacer obras derivadas

Bajo las condiciones siguientes:



Reconocimiento. Debe reconocer los créditos de la obra de la manera especificada por el autor o el licenciador (pero no de una manera que sugiera que tiene su apoyo o apoyan el uso que hace de su obra).



No comercial. No puede utilizar esta obra para fines comerciales.



Compartir bajo la misma licencia. Si altera o transforma esta obra, o genera una obra derivada, sólo puede distribuir la obra generada bajo una licencia idéntica a ésta.

- Al reutilizar o distribuir la obra, tiene que dejar bien claro los términos de la licencia de esta obra.
- Alguna de estas condiciones puede no aplicarse si se obtiene el permiso del titular de los derechos de autor
- Apart from the remix rights granted under this license, nothing in this license impairs or restricts the author's moral rights.

Advertencia

Los derechos derivados de usos legítimos u otras limitaciones reconocidas por ley no se ven afectados por lo anterior.
Esto es un resumen legible por humanos del texto legal (la licencia completa) disponible en los idiomas siguientes:

Catalán Castellano Euskera Gallego

Para ver una copia completa de la licencia, acudir a la dirección
<http://creativecommons.org/licenses/by-nc-sa/2.5/es/legalcode.es>

índice

(2.1) ¿qué es PHP?	5
(2.1.1) lenguajes de script de servidor	5
(2.1.2) PHP	5
(2.1.3) ventajas de PHP	6
(2.2) herramientas y software necesario	6
(2.3) bases de PHP	8
(2.3.1) etiqueta <? php?>	8
(2.3.2) HTML usa PHP y PHP usa HTML	9
(2.3.3) comentarios	9
(2.3.4) bases de escritura	9
(2.3.5) escribir en la salida	10
(2.4) variables	10
(2.4.1) introducción a las variables	10
(2.4.2) declarar	11
(2.4.3) predefinidas	11
(2.4.4) asignación de valores	11
(2.4.5) variables sin asignar valores	12
(2.4.6) tipos de datos	13
(2.4.7) referencias &	15
(2.4.8) constantes	15
(2.4.9) operadores	16
(2.5) estructuras de control	19
(2.5.1) sentencia if	19
(2.5.2) sentencia switch	22
(2.5.3) bucles	24
(2.6) uso de formularios HTML desde PHP	29
(2.6.1) repaso de los formularios HTML	29
(2.6.2) controles de HTML 5	36
(2.6.3) métodos de envío de datos del formulario	39
(2.6.4) recepción de datos de un formulario desde una página PHP	41
(2.6.5) usar la misma página para el formulario y la recepción	42

(2.7) funciones	43
(2.7.1) introducción	43
(2.7.2) declaración y uso de funciones personales	44
(2.7.3) alcance de las variables	45
(2.7.4) paso de parámetros por referencia	45
(2.7.5) parámetros predefinidos.....	47
(2.7.6) variables globales	47
(2.7.7) variables estáticas.....	48
(2.7.8) recursividad.....	48
(2.7.9) ámbito de las funciones	51
(2.8) inclusión de ficheros	51

(2)

introducción a PHP

(2.1) ¿qué es PHP?

(2.1.1) lenguajes de script de servidor

Las páginas web se crean mediante HTML, y este es un lenguaje muy limitado para atender a los requerimientos que actualmente se exigen. Por ello han aparecido numerosas extensiones al lenguaje que permiten enriquecer las páginas web.

Muchas mejoras están orientadas al cliente, es decir que se trata de código de otros lenguajes (llamados lenguajes de script) que se añaden al código HTML y que el ordenador que recibe la página debe interpretar a través del software apropiado. Por lo tanto el cliente de la página debe poseer el software apropiado. Y esto es un problema.

Por ello aparecieron lenguajes y mejoras en el lado del servidor. De modo que el programador añade al código HTML código de otro lenguaje script de la misma manera que el párrafo anterior. La diferencia es que este código no se le envía al cliente sino que es el servidor el que le interpreta. El cliente recibirá una página HTML normal y será el servidor el que traduzca el código script.

(2.1.2) PHP

Se trata indudablemente del lenguaje script de servidor más popular. Fue el primero en aparecer aunque realmente empezó a imponerse en torno al año 2000 por encima de ASP que era la tecnología de servidor reinante.

Hoy en día se puede instalar módulos para interpretar PHP en casi todos los servidores de aplicaciones web. En especial PHP tiene una gran relación con Apache.

Es un lenguaje basado en C y en Perl, que se ha diseñado pensando en darle la máxima versatilidad y facilidad de aprendizaje, por encima de la rigidez y coherencia semántica.

(2.1.3) ventajas de PHP

- (1) **Multiplataforma.** A diferencia de otros lenguajes (especialmente de **ASP** y **ColdFussion**), se trata de un lenguaje que se puede lanzar en casi todas las plataformas de trabajo (Windows, Linux, Mac,...)
- (2) **Abierto y gratuito.** Pertenece al software licenciado como **GNU**, la licencia del sistema Linux; lo que permite su distribución gratuita y que la comunidad mejore el código.
- (3) **Gran comunidad de usuarios.** La popularidad de PHP, junto con la gran defensa que de él hacen los defensores del código abierto, permite tener una comunidad amplia y muy dinámica a la que acudir en caso de necesidad.
- (4) **Apache, MySQL.** Apache es el servidor web y de aplicaciones más utilizado en la actualidad. MySQL es el servidor de bases de datos relacionales más popular en Internet para crear aplicaciones web. Puesto que PHP tiene una gran relación y compatibilidad con ambos productos (está de hecho muy pensado para hacer tándem con ellos), esto se convierte en una enorme (y a veces determinante) ventaja.
- (5) **Extensiones.** Dispone de un enorme número de extensiones que permiten ampliar las capacidades del lenguaje, facilitando la creación de aplicaciones web complejas.
- (6) **¿Fácil?** Es un punto muy controvertido. Los programadores PHP entusiastas, defienden esta idea; es indudable además que fue uno de los objetivos al crear este lenguaje. Sin embargo Microsoft defiende con energía que ASP permite crear aplicaciones web complejas con gran facilidad; y parece indudable que el lenguaje **ColdFussion** de Macromedia (ahora de Adobe) es más sencillo de aprender.

Las características de PHP correspondientes a la libertad de creación y asignación de valores a variables, tipos de datos poco restrictivos, y otras ausencias de reglas rígidas suelen ser los puntos que defienden los programadores de PHP para estimar su facilidad de aprendizaje. Sin embargo los programadores de lenguajes formales como C y Java, seguramente se encontrarán con más problemas que ventajas al aprender este lenguaje.

(2.2) herramientas y software necesario

Para crear aplicaciones web en PHP necesitamos disponer de software específico que complica los requisitos previos que tiene que tener la máquina en la que deseemos aprender y probar el lenguaje. Lo imprescindible es:

- **Un servidor web compatible con PHP.** La mayoría lo son, pero parece que la opción de usar Apache es la más recomendable por la buena relación que han tenido ambos productos. No obstante cada vez es más habitual utilizar PHP en servidores como **IIS** o **nginx**.
- **Motor PHP.** Se trata del software que extiende al servidor web anterior para conseguir que se convierta en un servidor de aplicaciones web PHP. Necesitamos descargar e instalar dicho motor correspondiente al módulo de PHP compatible con el servidor web con el que trabajemos.

- **IDE para PHP.** Un IDE es un entorno de desarrollo integrado; es decir, un software que facilita la escritura de código en otro lenguaje. En realidad se puede escribir código PHP en cualquier editor de texto (como el Bloc de Notas de Windows por ejemplo); pero estos entornos facilitan la edición de código (con coloreado especial de las palabras de PHP, corrección del código en línea, abreviaturas de código, plantillas,...) y su prueba y depuración.

Los IDEs más populares son:

- **Basados en Eclipse.** Eclipse es un entorno de programación de aplicaciones pensado para Java pero que dispone de muchas extensiones que permite programar para diferentes lenguajes, convirtiéndose así en la plataforma de programación más popular de la actualidad. Las extensiones de Eclipse más populares para programar en PHP son:
 - ▶ **Aptana.** Sea la versión sólo para PHP o la completa **Studio Pro** con posibilidad de usar en varios lenguajes es una de las más populares. Muy famosa para crear código **Javascript**, se ha adaptado con grandes resultados para PHP ya que permite casi todo lo necesario sobre este lenguaje.
 - ▶ **PHP Designer Toolkit (PDT).** Considerada la extensión oficial de Eclipse para programar en PHP, es quizá la más utilizada en la actualidad.
 - ▶ **Zend Studio.** Dispone de un **framework** (una plataforma) muy famosa para crear PHP usando plantillas. Su entorno compite con los anteriores en prestaciones.
- **Netbeans.** Se trata del entorno libre de programación competidor con Eclipse. Ambos se crearon para programar en Java, pero han extendido su uso a otros lenguajes. **NetBeans** dispone de una extensión para PHP, incluso se puede descargar una versión ligera de **NetBeans** sólo para PHP. Es una de las mejores opciones para programar en PHP.
- **Adobe Dreamweaver.** Se trata del software comercial de creación de páginas web más famoso del planeta. Tiene capacidad para escribir código PHP e incluso facilidades gráficas para hacerlo. Es inferior a los anteriores en cuanto a escritura de código, pero muy superior cuando queremos concentrarnos en el diseño del sitio.
- **Microsoft Expression Web.** Software comercial competidor del anterior (aunque por ahora mucho menos popular) y con capacidad de usar con PHP.
-
- **Depurador PHP.** Se trata de un software que se añade al módulo PHP para darle la capacidad de depurar el código. Los más utilizados por su potencia son **Zend Debugger** y **XDebug**.

(2.3) bases de PHP

El código PHP se escribe en un documento de texto al que se debe indicar la extensión **.php**. En realidad se trata de una página web HTML (o XHTML) normal a la que se le añaden etiquetas especiales.

(2.3.1) etiqueta `<? php?>`

Cuando en un documento web queremos añadir código **php** se indica por esta etiqueta:

```
<?php
    ...código PHP
?>
```

El código PHP se coloca en la zona de la página web donde más nos interese hacerlo. Así un primer documento PHP podría ser (suponiendo que incrustamos PHP en un documento de tipo XHTML 1.0 estricto):

```
<?php
    echo '<?xml version="1.0" encoding="UTF-8"?>'
?>

<!DOCTYPE html
    PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="es" lang="es">
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
        <title>Título</title>
    </head>
    <body>
        <?php
            echo "Hola";
        ?>
    </body>
</html>
```

El primer código PHP es necesario porque actualmente se consideran las páginas web XHTML como un documento XML más y por ello deben de llevar la cabecera `<?xml versión="1.0" encoding="utf-8"?>` obligatoria en todo documento XML. EL problema es que ese código en el servidor web daría problemas porque podría entender que es código PHP (al empezar igual que la etiqueta `<?php`) e intentaría traducirlo. Por ello es mejor escribirlo con la función `echo` para explicar que es código del documento final HTML y no PHP.

(2.3.2) HTML usa PHP y PHP usa HTML

Como se ha visto anteriormente el código PHP se incrusta dentro del código HTML. Lo interesante es que se puede incrustar código HTML en el PHP y tendría sentido ya que las funciones de escritura (**echo** y **print**) en realidad escriben hacia el resultado final, que en realidad es una página HTML por lo que se puede hacer algo como:

```
<?php
echo "Mi nombre es <strong>Jorge</strong>";
?>
```

(2.3.3) comentarios

Dentro del código PHP se pueden hacer tres tipos de comentario:

- **Comentarios de varias líneas.** Al igual que en lenguaje C, comienzan por **/*** y terminan por ***/**. Ejemplo:

```
<?php
/*
    Soy un comentario de
    varias líneas
*/
$x=10;
?>
```

- **Comentarios de una línea estilo C++.** Se ponen tras la barra doble **//**. Ejemplo:

```
<?php
$x=10; //esto es un comentario
?>
```

- **Comentarios de una línea estilo ShellScript.** Se ponen tras la almohadilla:

```
<?php
$x=10; #esto es un comentario
?>
```

(2.3.4) bases de escritura

Las normas básicas para escribir lenguaje PHP se basan en los lenguajes padres de este, es decir C y Perl. Son:

- Todas las líneas de código deben de finalizar con un punto y coma
- Se puede agrupar el código en bloques que se escriben entre llaves
- Una línea de código se puede partir o sangrar (añadir espacios al inicio) a voluntad con el fin de que sea más legible, siempre y cuando no partamos una palabra o un valor.
- PHP obliga a ser estricto con las mayúsculas y las minúsculas en algunos casos como el nombre de las variables; sin embargo con las palabras reservadas del lenguaje no es estricto. Es decir PHP entiende que **WHILE**, **while** e incluso

while es lo mismo al ser una palabra reservada. Sin embargo *\$var* y *\$VAR* no son iguales al ser el nombre de una variable.

(2.3.5) escribir en la salida

Aunque hay muchas funciones de escritura (para escribir en lo que será la página final) las fundamentales son **echo** y **print**.

echo es la más utilizada y en realidad es un comando del lenguaje. Tras echo se pasa uno o más textos (más adelante diremos *expresiones de cadena*) que cuando son literales se escriben entre comillas. Si se usa más de un texto, se separan con comas:

```
<?php
echo "Primer texto ", "segundo texto"
?>
```

El texto saldrá seguido en la página. Hay una forma abreviada de usar echo que es:

```
<?= "Primer texto ", "segundo texto"
?>
```

Se trata de no usar la etiqueta **<?php** para el código PHP. Si usamos **<? =** estaremos indicando que dentro de esa etiqueta hay texto a sacar con echo. No se puede colocar código de otro tipo dentro de **<? =** (porque entendería que es texto a mostrar con echo).

print funciona casi igual, pero tiene dos importantes diferencias:

- Devuelve un valor verdadero o falso dependiendo de si se pudo escribir el texto o no (en código complejo es muy útil)
- No admite varios textos, sólo uno. Aunque se le puede encadenar con el operador punto (.):

```
print "Primer texto ", "segundo texto"; //error
print "Primer texto ". "segundo texto"; //arreglado
```

(2.4) variables

(2.4.1) introducción a las variables

En todos los lenguajes de programación (y PHP no es una excepción) Las variables son contenedores que sirven para almacenar los datos que utiliza un programa. Dicho más sencillamente, son nombres que asociamos a determinados datos. La realidad es que cada variable ocupa un espacio en la memoria RAM del servidor que ejecute el código para almacenar el dato al que se refiere. Es decir cuando utilizamos el nombre de la variable realmente estamos haciendo referencia a un dato que está en memoria.

Las variables tienen un nombre (un **identificador**) que tiene que cumplir estas reglas:

- Tiene que empezar con el símbolo **\$**. Ese símbolo es el que permite distinguir a una variable de otro elemento del lenguaje PHP.
- El segundo carácter puede ser el guion bajo (**_**) o bien una letra.
- A partir del tercer carácter se pueden incluir números, además de letras y el guion bajo
- No hay límite de tamaño en el nombre

- Por supuesto el nombre de la variable no puede tener espacios en blanco (de ahí la posibilidad de utilizar el guión bajo)

Es conveniente que los nombres de las variables indiquen de la mejor forma posible su función. Es decir: `$saldo` es un buen nombre, pero `$x123` no lo es aunque sea válido.

También es conveniente poner a nuestras variables un nombre en minúsculas. Si consta de varias palabras el nombre podemos separar las palabras con un guión bajo en vez del espacio o empezar cada nueva palabra con una mayúscula. Por ejemplo: `$saldo_final` o `$saldoFinal`.

(2.4.2) declarar

La primera sorpresa para los programadores de lenguajes estructurados es que en PHP no es necesario declarar una variable. Simplemente se utiliza y ya está. Es decir si queremos que la variable `$edad` valga `15`, haremos:

```
$edad=15;
```

Y no será necesario indicar de qué tipo es esa variable. Esto es tremendamente cómodo pero también nos complica tremendamente la tarea de depurar nuestros programas al no ser nada rígido el lenguaje y permitir casi todo.

(2.4.3) predefinidas

El servidor web que aloje las páginas PHP pone a disposición del programador variables de sistema ya definidas para su uso en el programa. La mayoría son simplemente informativas. Todas suelen llevar el símbolo de subrayado en el segundo carácter además de escribirse en mayúsculas. La mayoría son arrays (se explican más adelante). Ejemplo:

```
echo $_SERVER["SERVER_PORT"];
```

Escribirá el número de puerto por el que se comunica el servidor web. Para mostrar todas las variables predefinidas en el servidor podemos escribir este código dentro de una página PHP

```
echo "<pre>";
print_r($_SERVER);
echo "</pre>";
```

Las etiquetas `pre`, permite que se muestra la información de una forma más atractiva (porque respeta los cambios de línea y tabulaciones devueltos por `print_r`). Por su parte `print_r` es una función para mostrar el contenido de los arrays.

(2.4.4) asignación de valores

Esta operación consiste en dar valor a una variable y se realiza con el símbolo `=`. Ejemplo:

```
$x=12;
```

Como se comentó anteriormente en PHP no es necesario declarar una variable antes de su uso. En el caso de asignar números se escriben tal cual (como en el ejemplo), los decimales se separan con el punto decimal. Los textos se encierran entre comillas

(simples o dobles, aunque se aconseja usar las dobles salvo cuando nos venga mejor las simples). Ejemplo:

```
$nombre="Anselmo";
```

Se pueden asignar resultados de aplicar fórmulas mediante operadores, como:

```
$beneficios=($gastos-$ingresos) * 0.6;
```

Otra cosa sorprendente es que una variable puede cambiar el tipo de datos que almacena, por ejemplo es válido:

```
$x=18; //asignamos un número
$x="Hola"; /asignamos un texto
```

(2.4.5) variables sin asignar valores

Un problema surge cuando queremos utilizar variables a las que no se les ha asignado ningún valor. Como:

```
<?php
echo $x; //es la primera vez que se usa $x. Error
$y=$x+2; //error, aunque y valdrá 2
echo $y;
?>
```

Ocurrirá un error al hacer ese uso de la variable. Aunque en realidad la directiva del archivo **php.ini**, **error_reporting** permite modificar los errores de aviso que se lanzan. Si bajamos su nivel de aviso, no detectará estos fallos. También podemos usar esa función al inicio de nuestro código para indicar fallos. Sería:

```
error_reporting(E_ALL); //avisa de todos los errores
```

Si deseamos que no nos avise de estos fallos. Habría que pasar a **error_reporting** otra constante; por ejemplo **E_USER_ERROR** avisa sólo si hay errores de nivel usuario (o más graves) en el código

A las variables sin declarar se les da valores por defecto, que dependerán del contexto en que se usen. Hay una función llamada **isset** a la que se le pasa un variable e indica si la variable tenía asignado un valor; en cualquier otro caso devuelve falso. Ejemplo:

```
<?php
echo isset($x); //escribe falso, o sea no escribe nada
$x=12;
echo isset($x); //devuelve verdadero, escribe 1
?>
```

(2.4.6) tipos de datos

enteros

A las variables se les puede asignar valores enteros. Los números enteros se usan tal cual. Pueden ser positivos o negativos:

```
$n1=17;  
$n2=-175;
```

Se puede usar sistema octal si se coloca un cero antes de la cifra entera:

```
$octal=071;  
echo $octal; //escribe 56
```

Además pueden estar en sistema hexadecimal si a la cifra se la antecede de un cero y una equis:

```
$hexa=0xA2BC;  
echo $hexa; //escribe 41660
```

coma flotante

Los números decimales en PHP son de tipo coma flotante. Este es un formato decimal para máquinas digitales que se manejan muy rápido por parte de un ordenador, ocupan poco en memoria, pero desgraciadamente no son exactos. Eso provoca que pueda haber algunos decimales que se pierdan.

Para asignar decimales hay que tener en cuenta en PHP que el formato es el inglés, por lo que las cifras decimales se separan usando un punto como separador decimal. Además es posible usar notación científica. Ejemplos:

```
$n1=234.12;  
$n2=12.3e-4; //eso es 12,3·10-4, es decir 0,00123
```

Los números decimales en coma flotante de PHP son equivalentes al formato **double** del lenguaje C.

cadena

Se denomina así a los textos, que en programación se les denomina cadenas de caracteres o **Strings**. Se asignan a las variables entrecomillando (en simples o dobles) el texto a asignar. Ejemplo:

```
$nombre="Jorge Sánchez";
```

Si el propio texto lleva comillas, se puede utilizar combinaciones de las comillas para evitar el problema. Por ejemplo:

```
$frase = 'Antonio dijo "Hola" al llegar';
```

Como queremos almacenar en **\$frase** el texto con **Hola** entre comillas, entonces englobamos todo el texto con comillas simples.

Otra opción es usar caracteres especiales, concretamente:

secuencia de escape	significado
\t	Tabulador
\n	Nueva línea

secuencia de escape	significado
<code>\f</code>	Alimentación de página
<code>\r</code>	Retorno de carro
<code>\"</code>	Dobles comillas
<code>\'</code>	Comillas simples
<code>\\</code>	Barra inclinada (<i>backslash</i>)
<code>\\$</code>	Símbolo dólar

Y así podremos:

```
$frase = "Antonio dijo \"Hola\" al llegar";
```

Las secuencias de escape sólo funcionan si están encerradas entre comillas dobles. Es decir, no funciona:

```
$frase = 'Antonio dijo \"Hola\" al llegar';
```

Las barras saldrán por pantalla también.

Un hecho de PHP muy interesante es que en un texto se puede incluir el valor de una variable. Por ejemplo:

```
$días=15;  
$texto="Faltan $días días para el verano";  
echo $texto; //escribe: Faltan 15 días para el verano
```

Por lo que el símbolo **\$** sólo se puede asignar a un texto si se usa su secuencia de escape `\$`.

Los textos se pueden concatenar con ayuda del operador punto (.). Ejemplo:

```
$nombre="Jorge";  
$apellidos="Sánchez Asenjo";  
echo "Nombre completo: ".$nombre." ".$apellidos;  
//escribe Nombre completo: Jorge Sánchez Asenjo
```

booleanos

Sólo pueden tomar como valores **TRUE** (verdadero) o **FALSE** (falso);

```
$verdadero=True;  
echo $verdadero; //escribe 1
```

Sorprende que echo devuelva el valor uno; la explicación es que True está asociado a valores positivos, mientras que False se asocia al cero. En concreto hay una relación todos los tipos de datos:

- Enteros: cero=False, resto=True
- Coma flotante: 0.0=False, resto=True
- Cadenas: False si están vacías
- Arrays: False si no posee ningún elemento
- Recurso: False si el recurso no es válido.

conversiones

En PHP como las variables pueden cambiar de tipo cuando se nos antoje, resulta que tiene que intentar que todas las expresiones tengan sentido y así, este código:

```
$v1=18;  
$v2="3 de Diciembre";  
echo $v1+$v2;
```

Escribe 21 (suma el 18 y el tres). Pero sin embargo:

```
$v1=18;  
$v2="3 de Diciembre";  
echo $v1.$v2;
```

Escribe **183 de Diciembre**.

No obstante se pueden convertir de forma forzosa los valores al tipo deseado; de esta forma elegiremos nosotros cómo realizar las conversiones. Se trata del habitual **operador de casting** del lenguaje C. Ejemplo:

```
$x=2.5;  
$y=4;  
$z=(int)$x * $y;
```

\$z vale 8 al convertir **\$x** en un entero. Posibilidades:

- **(int)** o **(integer)**. Convierte a entero
- **(real)**, **(double)** o **(float)**. Convierte a coma flotante
- **(string)**. Convierte a forma de texto
- **(array)**. Convierte a forma de array.
- **(object)**. Convierte a un objeto-

(2.4.7) referencias &

Es una de las claves de la programación en cualquier lenguaje. Se trata de variables que sirven para modificar otras variables existentes. Es decir, son variables que en lugar de almacenar valores, almacenan la dirección de otra variable. No es una copia de la otra variable, más bien es un sinónimo de la otra variable. Su uso principal aparece cuando se utilizan funciones. Ejemplo de uso:

```
$nombre="Antonio";  
$ref=&$nombre; //ref es una referencia a la variable $nombre  
echo $ref,"<br />"; //escribe Antonio  
$ref="Marisa";  
echo $nombre,"<br />" //escribe Marisa, a través de la referencia se ha  
                          // cambiado el nombre
```

(2.4.8) constantes

Las constantes almacenan valores que no cambian en el tiempo. La forma de definir constantes es gracias a la función **define**. Que funciona indicando el nombre que tendrá la constante, entre comillas, y el valor que se le otorga.

Ejemplo:

```
define("PI",3.141592);
```

Las constantes no utilizan el signo \$ de las variables, simplemente utilizan el nombre. Es decir escribir el valor de la constante PI tras haberla declarado con la instrucción anterior, sería:

```
echo PI;
```

Desde la versión 5.3 de PHP es posible definir una constante de una forma más estructurada. Se trata de utilizar la palabra clave **const** habitual en la mayoría de lenguajes de programación. Ejemplo de uso:

```
const PI=3.141592;
```

Aunque no es obligatorio, es conveniente usar mayúsculas para las constantes para diferenciarlas de las variables (aunque en PHP el signo dólar \$ ya hace esa diferenciación) ya que se considera una norma de buen estilo de escritura en cualquier lenguaje de programación.

(2.4.9) operadores

Lo habitual al programar en PHP es utilizar expresiones que permiten realizar comprobaciones o cálculos. Las expresiones dan un resultado que puede ser de cualquiera de los tipos de datos comentados anteriormente (enteros, decimales, booleanos, strings,...)

aritméticos

Son:

operador	significado
+	Suma
-	Resta
*	Producto
/	División
%	Módulo (resto)

Ejemplo de uso:

```
$x=15.5;  
$y=2;  
echo $x+$y,"<br />"; //escribe 17.5  
echo $x-$y,"<br />"; // escribe 13.5  
echo $x*$y,"<br />"; // escribe 31  
echo $x/$y,"<br />"; //escribe 7.75  
echo $x%$y,"<br />"; //escribe 1, sólo coge la parte entera
```

operadores condicionales

Sirven para comparar valores. Siempre devuelven valores booleanos. Son:

operador	significado
<	Menor
>	Mayor
>=	Mayor o igual
<=	Menor o igual
==	Igual, devuelve verdadero si las dos expresiones que compara son iguales
===	Equivalente, devuelve verdadero si las dos expresiones que compara son iguales y además del mismo tipo
!=	Distinto
!	No lógico (NOT)
&&	"Y" lógico
AND	"Y" lógico
	"O" lógico
OR	"O" lógico
XOR	"OR" exclusivo

Los operadores lógicos (**AND**, **OR** y **NOT**), sirven para evaluar condiciones complejas. NOT sirve para negar una condición. Ejemplo:

```
$edad = 21;
$mayorDeEdad = $edad >= 18; //mayorDeEdad será true
$menorDeEdad = !$mayorDeEdad; //menorDeEdad será false
echo $mayorDeEdad."\t".$menorDeEdad;
```

El operador **&&** (**AND**) sirve para evaluar dos expresiones de modo que si ambas son ciertas, el resultado será **true** sino el resultado será **false**. Ejemplo:

```
$carnetConducir=TRUE;
$edad=20;
$puedeConducir= ($edad>=18) && $carnetConducir;
//puedeConducir es TRUE, puesto que edades mayor de 18
//y carnet es TRUE
```

El operador **||** (**OR**) sirve también para evaluar dos expresiones. El resultado será **true** si al menos uno de las expresiones es **true**. Ejemplo:

```
$nieva =TRUE;
$lleue=FALSE;
$graniza=FALSE;
$malTiempo= $nieva || $lleue || $graniza; //malTiempo es TRUE porque nieva
```

La diferencia entre la igualdad y la equivalencia se puede explicar con este ejemplo:

```
$resultado= (18 == 18.0); //resultado es verdadero  
$resultado= (18 === 18.0); //resultado es falso
```

En el ejemplo `18===18.0` devuelve falso porque aunque es el mismo valor, no es del mismo tipo.

operadores de asignación

Ya se ha comentado el operador de asignación que sirve para dar valor a una variable. Ejemplo:

```
$x=$y+9.8;
```

Sin embargo existen operadores que permiten mezclar cálculos con la asignación, ejemplo:

```
$x += 3;
```

En el ejemplo anterior lo que se hace es sumar `3` a la `x` (es lo mismo `$x+=3`, que `$x=$x+3`). Eso se puede hacer también con todos estos operadores:

<code>+=</code>	<code>-=</code>	<code>*=</code>	<code>/=</code>
<code>&=</code>	<code> =</code>	<code>^=</code>	<code>%=</code>
<code>>>=</code>	<code><<=</code>	<code>.=</code>	

Otros operadores de asignación son “++” (incremento) y “--” (decremento). Ejemplo:

```
$x++; //esto es $x=$x+1;  
$x--; //esto es $x=$x-1;
```

Pero hay dos formas de utilizar el incremento y el decremento. Se puede usar por ejemplo `x++` o `++x`

La diferencia estriba en el modo en el que se comporta la asignación. Ejemplo:

```
$x=5;  
$y=5;  
$z=$x++; //z vale 5, x vale 6  
$z=++$y; //z vale 6, y vale 6
```

operadores de BIT

Manejan números, a los cuales se les pasa a formato binario y se opera con ellos BIT a BIT. Son:

operador	significado
<code>&</code>	AND
<code> </code>	OR
<code>~</code>	NOT
<code>^</code>	XOR
<code>>></code>	Desplazamiento a la derecha
<code><<</code>	Desplazamiento a la izquierda

concatenación

El punto (.) es un operador que permite unir textos. Su uso es muy sencillo. Ejemplo:

```
$x="Hola ";  
$y="a todo el mundo";  
$z=$x.$y; //$z vale "Hola a todo el mundo"
```

Hay operador de asignación y concatenación de texto, se trata del operador .=. Ejemplo de uso:

```
$x="Hola ";  
$x.="a todo el mundo";  
echo $x; //escribe "Hola a todo el mundo"
```

También es válido el uso de la concatenación en la asignación. Ejemplo:

```
$texto = "Hola ";  
$texto.= "amigos y ";  
$texto.="amigas"; //$texto es Hola amigos y amigas
```

(2.5) estructuras de control

Hasta ahora las instrucciones que hemos visto, son instrucciones que se ejecutan secuencialmente; es decir, podemos saber lo que hace el programa leyendo las líneas de izquierda a derecha y de arriba abajo.

Las instrucciones de control de flujo permiten alterar esta forma de ejecución. A partir de ahora habrá líneas en el código que se ejecutarán o no dependiendo de una **condición**.

Esa condición se construye utilizando lo que se conoce como **expresión lógica**. Una expresión lógica es cualquier tipo de expresión que dé un resultado verdadero o falso.

Las expresiones lógicas se construyen a través de los operadores relacionales (==, >, <, ...) y lógicos (&&, ||, !, ...) vistos anteriormente.

(2.5.1) sentencia if

sentencia condicional simple

Se trata de una sentencia que, tras evaluar una expresión lógica, ejecuta una serie de instrucciones en caso de que la expresión lógica sea verdadera. Si la expresión tiene un resultado falso, no se ejecutará ninguna expresión. Su sintaxis es:

```
if(expresión lógica) {  
    instrucciones  
    ...  
}
```


Las llaves se requieren sólo si va a haber varias instrucciones. En otro caso se puede crear el **if** sin llaves:

```
if(expresión lógica) sentencia;
```

Ejemplo:

```
if($nota>=5){  
    echo "Aprobado";  
    $aprobados++;  
}
```

Sólo se escribe *Aprobado* si la variable *\$nota* es mayor o igual a cinco.

Hay otra posibilidad de sintaxis:

```
if(expresión lógica) :  
    instrucciones  
    ...  
endif;
```

En lugar de las llaves, cierra el bloque **if** con la palabra **endif**. Los dos puntos tras el paréntesis son obligatorios. Se trata de una forma simplificada (que, ciertamente, hay que usar con cuidado).

sentencia condicional compuesta

Es igual que la anterior, sólo que se añade un apartado **else** que contiene instrucciones que se ejecutarán si la expresión evaluada por el **if** es falsa. Sintaxis:

```
if(expresión lógica){  
    instrucciones a ejecutar si la expresión es verdadera  
    ...  
}  
else {  
    instrucciones a ejecutar si la expresión es falsa  
    ...  
}
```

Como en el caso anterior, las llaves son necesarias sólo si se ejecuta más de una sentencia en el bloque.

Ejemplo de sentencia **if-else**:

```
if($nota>=5){  
    echo "Aprobado";  
    $aprobados++;  
}  
else {  
    echo "Suspenso";  
    suspensos++;  
}
```

La forma simplificada de esta sentencia sería:

```
if($nota>=5):  
    echo "Aprobado";  
    $aprobados++;  
else :  
    echo "Suspenso";  
    $suspensos++;  
endif;
```

No se usan las llaves, en su lugar se usan los dos puntos y el cierre con **endif**.

sentencia condicional múltiple

Permite unir varios **if** en un solo bloque. Ejemplo:

```
if($nota<5){  
    echo "suspenso";  
    $suspensos++;  
}  
elseif($nota<7){  
    echo "aprobado";  
    $aprobados++;  
}  
elseif($nota<9){  
    echo "notable";  
    $aprobados++;  
}  
else{  
    echo "sobresaliente";  
    $aprobados++;  
}
```

Cada sentencia **elseif** permite añadir una condición que se examina si la anterior no es verdadera. En cuanto se cumpla una de las expresiones, se ejecuta el código del **if** o **elseif** correspondiente. EL bloque **else** se ejecuta si no se cumple ninguna de las condiciones anteriores.

La versión simplificada sería:

```
if($nota<5):  
    echo "suspense";  
    $suspensos++;  
elseif($nota<7):  
    echo "aprobado";  
    $aprobados++;  
elseif($nota<9):  
    echo "notable";  
    $aprobados++;  
else:  
    echo "sobresaliente";  
    $aprobados++;  
}
```

(2.5.2) sentencia switch

```
switch (expresión) {  
    case valor1:  
        instrucciones del valor 1  
        [break]  
    [case valor2:  
        instrucciones del valor 1  
        [break]  
    [  
        .  
        .  
        .]  
    [default:  
        instrucciones que se ejecutan si la expresión no toma  
        ninguno de los valores anteriores  
        ..]  
}
```

Los corchetes indican que su contenido es opcional. Es decir, los **break** son opcionales y la cláusula default también.

Esta instrucción se usa cuando tenemos instrucciones que se ejecutan de forma diferente según evaluemos el conjunto de valores posible de una expresión. Cada **case** contiene un valor de la expresión; si efectivamente la expresión equivale a ese valor, se ejecutan las instrucciones de ese **case** y de los siguientes.

La instrucción **break** se utiliza para salir del **switch**. De tal modo que si queremos que para un determinado valor se ejecuten las instrucciones de un apartado **case** y sólo las de ese apartado, entonces habrá que finalizar ese **case** con un **break**.

El bloque **default** sirve para ejecutar instrucciones para los casos en los que la expresión no cumpla ninguno de los valores anteriores. Ejemplo:

```
srand(time());
$diaSemana=rand(1,7);
switch ($diaSemana) {
    case 1:
        $dia="Lunes";
        break;
    case 2:
        $dia="Martes";
        break;
    case 3:
        $dia="Miércoles";
        break;
    case 4:
        $dia="Jueves";
        break;
    case 5:
        $dia="Viernes";
        break;
    case 6:
        $dia="Sábado";
        break;
    case 7:
        $dia="Domingo";
        break;
    default:
        $dia="?";
}
```

No usar break para salir del switch puede servir para cosas como:

```
srand(time());
$diaSemana=rand(1,7);
switch ($diaSemana) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        echo "Laborable";
        break;
    case 6:
    case 7:
        echo "Festivo";
        break;
    default:
        echo "?";
}
```

Hay una versión simplificada de switch que permite usar los dos puntos al final de la línea del switch y sustituir la llave final del switch por el texto **endswitch**.

(2.5.3) bucles

Un bucle es un conjunto de sentencias que se repiten mientras se cumpla una determinada condición. Los bucles agrupan instrucciones las cuales se ejecutan continuamente hasta que una determinada condición que se evalúa sea falsa.

bucle while

Sintaxis:

```
while (expresión lógica) {
    sentencias que se ejecutan si la condición es true
}
```

El programa se ejecuta siguiendo estos pasos:

- (1) Se evalúa la expresión lógica
- (2) Si la expresión es verdadera ejecuta las sentencias, sino el programa abandona la sentencia **while**
- (3) Tras ejecutar las sentencias, volvemos al paso 1

Ejemplo (escribir números del 1 al 100):

```
$i=1;  
while ($i<=100){  
    echo $i."<br />";  
    $i++;  
}
```

while tiene también una versión simplificada. El ejemplo anterior quedaría:

```
$i=1;  
while ($i<=100):  
    echo $i."<br />";  
    $i++;  
endwhile;
```

Este tipo de bucles son los fundamentales. Todas las demás instrucciones de bucle se basan en el bucle **while**. Por ello se comentan con él los dos principales tipos de bucle: de centinela y de contador.

Otra sintaxis posible es:

```
while (expresión lógica) :  
    sentencias que se ejecutan si la condición es true  
endwhile;
```

bucles de contador

Se llaman así a los bucles que se repiten una serie determinada de veces. Dichos bucles están controlados por un contador (o incluso más de uno). El contador es una variable que va variando su valor (de uno en uno, de dos en dos,... o como queramos) en cada vuelta del bucle. Cuando el contador alcanza un límite determinado, entonces el bucle termina.

En todos los bucles de contador necesitamos saber:

- (1) Lo que vale la variable contadora al principio. Antes de entrar en el bucle
- (2) Lo que varía (lo que se incrementa o decrementa) el contador en cada vuelta del bucle.
- (3) Las acciones a realizar en cada vuelta del bucle
- (4) El valor final del contador. En cuanto se rebase el bucle termina. Dicho valor se pone como condición del bucle, pero a la inversa; es decir, la condición mide el valor que tiene que tener el contador para que el bucle se repita y no para que termine.

Ejemplo:

```
$i=10; /*Valor inicial del contador, empieza valiendo 10
      (por supuesto i debería estar declarada como entera, int) */

while ($i<=200){ /* condición del bucle, mientras i sea menor de
                200, el bucle se repetirá, cuando i rebase este
                valor, el bucle termina */

    echo $i."<br />"; /*acciones que ocurren en cada vuelta del bucle
                    en este caso simplemente escribe el valor
                    del contador */

    $i+=10; /* Variación del contador, en este caso cuenta de 10 en 10*/
}
/* Al final el bucle escribe:
10
20
30
...
y así hasta 200
*/
```

bucles de centinela

Es el segundo tipo de bucle básico. Una condición lógica llamada **centinela**, que puede ser desde una simple variable booleana hasta una expresión lógica más compleja, sirve para decidir si el bucle se repite o no. De modo que cuando la condición lógica se incumpla, el bucle termina.

Esa expresión lógica a cumplir es lo que se conoce como centinela y normalmente la suele realizar una variable booleana.

Ejemplo:

```
$salir=FALSE; /* En este caso el centinela es una variable
               booleana que inicialmente vale falso */

while($salir==FALSE){ /* Condición de repetición: que salir siga siendo
                       falso. Ese es el centinela.
                       También se podía haber escrito simplemente:
                       while(!$salir)
                       */
    $n=rand(1,500); // Lo que se repite en el
    echo($n); /* bucle: calcular un número
               aleatorio de 1 a 500 y escribirlo */
    $salir=($n%7==0); /* El centinela vale verdadero si el número es
                       múltiplo de 7
                       */
}
```

Comparando los bucles de centinela con los de contador, podemos señalar estos puntos:

- Los bucles de contador se repiten un número concreto de veces, los bucles de centinela no
- Un bucle de contador podemos considerar que es seguro que finalice, el de centinela puede no finalizar si el centinela jamás varía su valor (aunque, si está bien programado, alguna vez lo alcanzará)
- Un bucle de contador está relacionado con la programación de algoritmos basados en series.

Un bucle podría ser incluso mixto: de centinela y de contador. Por ejemplo imaginar un programa que escriba números de uno a 500 y se repita hasta que llegue un múltiplo de 7, pero que como mucho se repite ocho veces.

Sería:

```
$salir = FALSE; //centinela
$i=1; //contador
while ($salir == false && $i<=8) {
    $n = rand(1,500);
    echo $n."<br />";
    $i++;
    $salir = ($n % 7 == 0);
}
```

do while

La única diferencia respecto a la anterior está en que la expresión lógica se evalúa después de haber ejecutado las sentencias. Es decir el bucle al menos se ejecuta una vez. Es decir los pasos son:

- (1) Ejecutar sentencias
- (2) Evaluar expresión lógica
- (3) Si la expresión es verdadera volver al paso 1, sino continuar fuera del while

Sintaxis:

```
do {  
    instrucciones  
} while (expresión lógica)
```

Ejemplo (contar de uno a 1000):

```
$i=0;  
do {  
    $i++;  
    echo $i."<br />";  
} while ($i<1000);
```

Se utiliza cuando sabemos al menos que las sentencias del bucle se van a repetir una vez (en un bucle **while** puede que incluso no se ejecuten las sentencias que hay dentro del bucle si la condición fuera falsa, ya desde un inicio).

De hecho cualquier sentencia **do..while** se puede convertir en while. El ejemplo anterior se puede escribir usando la instrucción while, así:

```
int i=0;  
$i++;  
echo $i."<br />";  
while (i<1000) {  
    $i++;  
    echo $i."<br />";  
}
```

for

Es un bucle más complejo especialmente pensado para rellenar arrays o para ejecutar instrucciones controladas por un contador. Una vez más se ejecutan una serie de instrucciones en el caso de que se cumpla una determinada condición. Sintaxis:

```
for(inicialización;condición;incremento){  
    sentencias  
}
```

Segunda versión:

```
for(inicialización;condición;incremento):  
    sentencias  
endfor;
```

Las sentencias se ejecutan mientras la condición sea verdadera. Además antes de entrar en el bucle se ejecuta la instrucción de inicialización y en cada vuelta se ejecuta el incremento. Es decir el funcionamiento es:

- (1) Se ejecuta la instrucción de inicialización
- (2) Se comprueba la condición
- (3) Si la condición es cierta, entonces se ejecutan las sentencias. Si la condición es falsa, abandonamos el bloque *for*
- (4) Tras ejecutar las sentencias, se ejecuta la instrucción de incremento y se vuelve al paso 2

Ejemplo (contar números del 1 al 1000):

```
for($i=1;$i<=1000;$i++){  
    echo $i."<br />";  
}
```

La ventaja que tiene es que el código se reduce. La desventaja es que el código es menos comprensible. El bucle anterior es equivalente al siguiente bucle **while**:

```
$i=1; /*sentencia de inicialización*/  
while($i<=1000) { /*condición*/  
    echo $i."<br />";  
    $i++; /*incremento*/  
}
```

(2.6) uso de formularios HTML desde PHP

Los formularios son el elemento de las páginas web que permiten recoger valores del usuario y enviarlos a una dirección URL para que se les procese. En nuestro caso la URL será una página PHP que recogerá los valores introducidos en el formulario y actuará en consecuencia.

(2.6.1) repaso de los formularios HTML

etiqueta **form**

Todo formulario HTML comienza con una etiqueta **form**, dentro de ella se colocan todos los controles del formulario.

Fundamentalmente la etiqueta **form** posee dos atributos:

- **method**. Que permite indicar qué instrucción http utilizaremos para pasar la información al destino de nuestro formulario. Las opciones habituales son **GET** y **POST**. No se distingue entre mayúsculas y minúsculas.
- **action**. Es el atributo que contiene la URL de la página web que procesará el formulario.

Ejemplo:

```
<form action="control_form.php" method="GET">
...
</form>
```

cuadros de texto

Los cuadros de texto permiten recoger texto. Su sintaxis es:

```
<input type="text" name="nombre" />
```

Es saca en la página un cuadro en el que el usuario puede introducir texto. Además podemos utilizar estos atributos:

- **value**. Da un valor inicial al cuadro, se usa para indicar un texto de ayuda al relleno.
- **id**. Identificador del cuadro. La W3C (organismo que estandariza XML y HTML) recomienda usar **id** en lugar de **name**, pero lo cierto es que PHP recoge los valores gracias a **name**, no funciona con **id**. Por lo que para más seguridad los diseñadores suelen utilizar ambos atributos.

Ejemplo:

```
<form action="control_form.php" method="get">
  Escriba su nombre y apellidos<input type="text" name="nombre" />
</form>
```

Ese código da como resultado:

Escriba su nombre y apellidos

botón de envío

Sirve para llevar a cabo la comunicación entre el formulario y la página que recoge sus datos. La sintaxis es:

```
<input type="submit" value="texto del botón" />
```

En cuanto se pulsa este botón, los datos del resto de controles se envían a la página receptora del formulario (que se corresponde a la URL indicada en el apartado **action** de la etiqueta **form**).

botón de restablecer

La sintaxis de este botón es:

```
<input type="reset" value="texto del botón" />
```

Este tipo de botones lo que hacen es anular los datos que se han introducidos en los controles del formulario. Deja todos los controles en su estado inicial.

botón genérico

Un botón genérico se marca indicando `type="button"` en la etiqueta **type**. En los formularios no se usa para enviar o configurar la información, sino que se utiliza normalmente para capturar su pulsación (mediante **JavaScript** es lo más habitual) y responder en consecuencia.

cuadro de contraseñas

Funcionan como los cuadros de texto, sólo que el texto que se introduce se oculta, mostrando sólo puntos o asteriscos. La sintaxis es:

```
<input type="password" name="nombre" />
```

Usa los mismos atributos que los cuadros de texto

botones de radio

Se trata de un control fácil de usar que permite elegir una de entre varias opciones. Todas las opciones deben de tener el mismo nombre y sólo cambia el valor. Ejemplo:

```
<form action="control_form.php" method="get">
  <p>Sexo:</p>
  <input type="radio" name="sexo" value="hombre"/> Hombre<br />
  <input type="radio" name="sexo"
    value="mujer" checked="checked"/> Mujer<br />
</form>
```

El resultado es:

Sexo:

- ☐ Hombre
☒ Mujer

El atributo **checked** (que sólo admite el valor **checked**) hace que el botón en el que se usa, aparezca chequeado por defecto

casillas de verificación

Se usan igual que los botones de radio, aunque no se recomienda agrupar varios con el mismo nombre; no porque no se pueda, sino porque las casillas se asocian a valores que se activan o desactivan.

La diferencia es que el tipo (**type**) es **checkbox** en lugar de **radio**. también posee el atributo **checked** para que inicialmente la casilla aparezca pulsada.

Ejemplo:

```
<form action="control_form.php" method="get">
  <p>Sexo:</p>
  <input type="checkbox" name="sexo" value="hombre"/>Hombre<br />
  <input type="checkbox" name="sexo" value="mujer"
        checked="checked"/>Mujer<br />
  Estado civil: Casado <input type="checkbox" name="estadocivil"
        value="casado" checked="checked"/>
  <br />
  <input type="submit" value="Enviar" />
</form>
```

El resultado:

Sexo:

☐ Hombre

☒ Mujer

Estado civil: Casado ☒

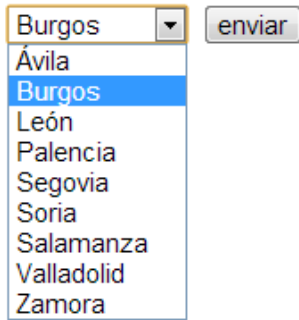
cuadros combinados

Un cuadro combinado permite el uso de una lista de opciones en la que se puede elegir la deseada. Todo cuadro comienza con una etiqueta **select** que es la encargada de dar nombre (**name**) al control. Dentro cada opción del cuadro se indica con una etiqueta **option** a la que se da valor mediante el atributo **value**. Dentro de la etiqueta **option** se coloca el texto que verá el usuario.

Ejemplo:

```
<form action="control_form.php" method="get">
  <select name="provincia">
    <option value="av">Ávila</option>
    <option value="bu">Burgos</option>
    <option value="l">León</option>
    <option value="p">Palencia</option>
    <option value="sg">Segovia</option>
    <option value="so">Soria</option>
    <option value="sa">Salamanca</option>
    <option value="va">Valladolid</option>
    <option value="za">Zamora</option>
  </select>
  <input type="submit" value="enviar"/>
</form>
```

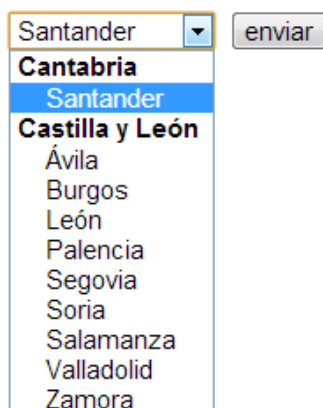
Resultado:



Se pueden agrupar opciones dentro del cuadro usando el elemento **optgroup**. Ejemplo:

```
<form action="control_form.php" method="get">
  <select name="provincia">
    <optgroup label="Cantabria">
      <option value="s">Santander</option>
    </optgroup>
    <optgroup label="Castilla y León">
      <option value="av">Ávila</option>
      <option value="bu">Burgos</option>
      <option value="l">León</option>
      <option value="p">Palencia</option>
      <option value="sg">Segovia</option>
      <option value="so">Soria</option>
      <option value="sa">Salamanca</option>
      <option value="va">Valladolid</option>
      <option value="za">Zamora</option>
    </optgroup>
  </select>
  <input type="submit" value="enviar"/>
</form>
```

Da como resultado:



cuadro de selección de archivo

Cuando se usa `type="file"` en una etiqueta `input`, entonces aparece un botón que al pulsarle hace aparecer un cuadro de selección de archivos mediante el cual podremos elegir un archivo. La ruta local a dicho archivo es lo que se guarda para ser enviado al sitio o página que recibe los valores del formulario. Ejemplo:

```
<form action="recogida1.php" method="get">  
  Elija el archivo  
  <input type="file" name="archivo" />  
  <input type="submit" value="enviar" /> <br />  
</form>
```

Resultado:

Elija el archivo No se ha seleccionado ningún archivo

Al pulsar `seleccionar archivo` aparece un cuadro, cuando elijamos el archivo aparece su nombre.

cuadro de texto multilinea

La etiqueta `textarea` permite colocar un cuadro de texto de varias líneas para que el usuario puede introducir un texto largo. El atributo `rows` permite indicar la altura en líneas de texto del cuadro y el atributo `cols`, la anchura en caracteres. Ejemplo:

```
<form action="control_form.php" method="get">  
  Escribe un texto descriptivo: <br />  
  <textarea rows="10" cols="40" name="texto"></textarea> <br />  
  <input type="submit" value="enviar" /> <br />  
</form>
```


Resulta:

Escribe un texto descriptivo:



enviar

Entre la etiqueta **textarea** se puede colocar texto que aparecerá inicialmente dentro del cuadro.

agrupación de controles

La etiqueta **fieldset** permite agrupar controles para que visualmente sea más cómodo el relleno de los controles. Visualmente los controles aparecerán recuadrados y se suele utilizar una etiqueta inmediatamente interior a **fieldset** que es **legend** que contiene el texto que encabezará al grupo de controles.

Ejemplo:

```
<form action="control_form.php" method="get">
  <fieldset >
    <legend>Datos personales</legend>
    <input type="radio" name="sexo" value="hombre"/> Hombre<br />
    <input type="radio" name="sexo" value="mujer"
      checked="checked"/> Mujer<br />
    Estado civil: Casado <input type="checkbox" name="estadocivil"
      value="casado" checked="checked"/>
    <br />
  </fieldset>
  <input type="submit" value="Enviar" />
</form>
```

El resultado:



(2.6.2) controles de HTML 5

El nuevo estándar HTML ha mejorado notablemente los formularios gracias a nuevos controles. La pega es que no todos los controles son soportados por los navegadores; hay controles que algunos navegadores no permiten utilizar. Cuando un navegador no comprende un control HTML 5, suele traducirlo como un cuadro de texto normal.

cuadros de texto especializados

La diferencia entre ellos es el uso del atributo **type**. Los buenos navegadores que reconocen estos cuadros, varían su uso para adaptarles a su función y así facilitar al usuario la entrada de datos. De hecho los navegadores validan los datos y no les dejan enviar hasta que sean correctos. Posibilidades:

- **input type="number"**. Acepta sólo números. El navegador se asegura de no aceptar texto. El atributo **maxlength** permite indicar un valor máximo para el cuadro. Si, por ejemplo, ponemos **maxlength="5"** y entonces sólo se aceptarán cinco caracteres.
- **input type="email"**. Acepta sólo direcciones de correo electrónico
- **input type="url"**. Acepta sólo direcciones URL.
- **input type="date"**. Acepta sólo fechas válidas. Usa el formato de fecha configurado en el sistema operativo del usuario que visita la página. Los navegadores además proporcionan un cuadro visual más sencillo para recoger la fecha.
- **input type="time"**. Acepta sólo horas válidas; funciona igual que el cuadro anterior.
- **input type="datetime"**. Acepta fecha y hora.
- **input type="month"**. Acepta sólo números del 1 al 12, referidos a un mes.
- **input type="search"**. Presenta un cuadro de texto pensado para hacer búsquedas.
- **input type="tel"**. Permite introducir números de teléfono.
- **input type="range"**. Presenta un control para elegir datos entre un rango. Los atributos **max** y **min** establecen el rango máximo y mínimo del control. El atributo **step** indica cuánto se mueve el control (si de uno en uno, de dos en dos,...).
- **input type="color"**. Presenta un control de selección de colores. El color se toma en formato **#xxxxxx** donde cada **x** es una cifra hexadecimal. Es decir lo toma en el formato habitual de colores de HTML.

etiquetado de controles

En lugar de poner la información de los controles poniendo texto directamente, se recomienda usar la etiqueta **label** que utiliza un atributo **for** que sirve para asociar el texto interno a la etiqueta **label** respecto al nombre del control que se indica con el atributo **for**.

Esto permite una mejor usabilidad (los navegadores reconocen **label** y, por ejemplo, al hacer clic en el texto de la etiqueta, el control asociado recibe el foco).

El ejemplo anterior usando **label** sería:

```
<form action="control_form.php" method="get">
  <label for="texto">Escribe un texto descriptivo:</label>
  <textarea rows="10" cols="40" name="texto" ></textarea><br />
  <input type="submit" value="enviar"/><br />
</form>
```

datalist

No hay muchos navegadores que acepten esta etiqueta, pero es muy potente permite añadir entradas a un control de cuadro de texto (y también a cuadros especializados como los de email, url,...). La forma de usar consiste en usar el atributo HTML5 **list** existente en la etiqueta **input**. Ese atributo asociará el cuadro de texto a la lista de valores. Luego dentro de **datalist** se colocan etiquetas **option** para cada opción en la lista.

Ejemplo:

```
<form action="recogida.php" method="get">
  <label for="gustos">
    Escribe en qué te gusta pasar tu tiempo de ocio
  </label>
  <input type="text" id="gustos" name="gustos" list="listaGustos" />
  <datalist id="listaGustos">
    <option label="deportes" value="Deportes" />
    <option label="teatro" value="Teatro" />
    <option label="cine" value="Cine" />
    <option label="leer" value="Leer" />
  </datalist>
  <input type="submit" value="enviar"/><br />
</form>
```

No aparece un cuadro combinado, será un cuadro de texto que permitirá que aparezca la lista de opciones. Pero podremos escribir lo que queramos.

Sin embargo esta forma no funciona en la mayoría de navegadores y por ello se usa un truco que hace que los navegadores no compatibles con HTML5 vean la lista como un cuadro combinado y los compatibles como un cuadro de texto en el que se podría escribir:

```
<form action="recogida.php" method="get">
  <label for="gustos">
    Escribe en qué te gusta pasar tu tiempo de ocio
  </label>
  <datalist id="listaGustos">
    <select name="listaGustos">
      <option value="deportes" >Deportes</option>
      <option value="teatro" >Teatro</option>
      <option value="cine" >Cine</option>
      <option value="leer" >Leer</option>
    </select>
  </datalist>
```

```
<input id="gustos" name="gustos" list="listaGustos" />
<input type="submit" value="enviar"/><br />
</form>
```

El truco consiste en meter **select** dentro del **datalist**, los navegadores modernos ignorarán la etiqueta **select** y los viejos el **datalist**.

atributo **required**

Este atributo obliga a rellenar con algún valor el control en el que se usa. Es decir hace que un determinado control sea de obligado rellenado en un formulario. Uso:

```
<input type="text" name="texto" id="texto" required />
```

O bien, usando una forma más compatible con XMLN:

```
<input type="text" name="texto" id="texto" required="required" />
```

atributo **multiple**

Permite en los cuadros de entrada de texto que el usuario pueda indicar varios valores si les separa con comas.

atributo **pattern**

Permite colocar una expresión regular en un cuadro de texto que, obligatoriamente, tendrá que cumplir el cuadro en el que se use el atributo. Ejemplo (cuadro de texto que sólo acepta introducir 5 letras mayúsculas y tres números):

```
<form action="recogida1.php" method="get">
<label for="texto">
  Escribe el nº de serie (5 letras y tres números)
</label>
<input type="text" pattern="[A-Z]{5}[0-9]{3}" id="gustos" name="texto"
/>
<input type="submit" value="enviar"/><br />
</form>
```

atributo **placeholder**

Un **placeholder** es un texto que ayuda a rellenar un cuadro de un formulario (está especialmente pensado para los cuadros de texto) colocando un texto inicial en el cuadro que se va en cuanto el cuadro obtiene el foco del usuario (porque, por ejemplo, el usuario le hace clic). Ejemplo:

```
<form action="recogida1.php" method="get">
<label for="texto">
  Escribe el nº de serie
</label>
<input type="text" pattern="[A-Z]{5}[0-9]{3}" id="gustos" name="texto"
placeholder="5 letras y tres números" />
<input type="submit" value="enviar"/><br />
</form>
```

El resultado es:

Escribe el nº de serie

atributo autocomplete

Permite activar (valor **on**) o desactivar (valor **off**) el autocompletado del navegador. El autocompletado es la opción que permite a los usuarios cuando rellenan un formulario ver entradas habituales que han escrito en el mismo u otros formularios. a veces conviene desactivar cuando lo que se escribe son datos sensibles (nombre de usuario, contraseñas).

atributos min,max y range

Son atributos que se pueden utilizar en muchos tipos de cuadros (**number**, **date**, **time**, **range**,..) que almacenan valores numéricos o similares. **min** y **max** establecen los límites del cuadro. Por ejemplo si el cuadro es numérico y ponemos **min=1** y **max=50** y eso impedirá poner valores en el cuadro fuera de esos topes. El parámetro **step** indica el mínimo incremento de valor en el cuadro; si en el ejemplo anterior ponemos **step=3** del valor **1** saltaremos al **4**. Los controles del cuadro (en el caso de los cuadros numéricos, las flechitas) obedecen a esa configuración.

(2.6.3) métodos de envío de datos del formulario

Como se ha comentado antes, un formulario puede enviar la información usando el atributo **method** del formulario y eligiendo **GET** o **POST**.

paso de datos mediante GET

El método GET lo que hace es añadir a la URL destinataria del formulario los datos recogidos del formulario. Ejemplo:

```
<form action="recogida1.php" method="get">
<label for="nombre">Escriba su nombre</label>
<input type="text" name="nombre" /> <br />
<label for="apellidos">Escriba sus apellidos</label>
<input type="text" name="apellidos" /> <br />
<label for="direccion">Escriba su dirección</label>
<input type="text" name="direccion" /> <br />
<label for="telefonos">Escriba sus teléfonos:</label><br />
<input type="text" name="telefono1" maxlength="9" pattern="[0-9]{9}" /> <br />
<input type="text" name="telefono2" maxlength="9" pattern="[0-9]{9}" /> <br />
<input type="submit" value="enviar" /><br />
</form>
```

El resultado del código, rellenando los campos, sería:

Escriba su nombre

Escriba sus apellidos

Escriba su dirección

Escriba sus teléfonos:

Suponiendo que estamos probando el código en la página web <http://localhost/pruebas/form11.html>, al pulsar enviar, se genera la URL:

```
http://localhost/pruebas/recogida1.php?nombre=Jorge&apellidos=S%C3%A1nchez+Asenjo&direccion=C%2F+Los+Vientos&telefono1=666666666&telefono2=999999999
```

El apellido **Sánchez** se convierte en **S%C3%A1nchez** por que se requiere codificar en Unicode y en las URL es necesario utilizar el operador **%**, al que se le pasa el código hexadecimal del carácter. En este caso hay dos, porque en **utf-8** el símbolo **á** usa dos bytes.

Dejando de lado la cuestión de la codificación, la forma de una URL tipo GET es:

```
http://urlpágina?var1=valor1&var2=valor2&....
```

Es decir, usa pares entre el nombre de la variable (en el formulario asignada a través del atributo **name**) y el valor que se le dio en el formulario. Cuando un control del formulario se queda sin valor, entonces la variable se queda sin definir y, por lo tanto, la página PHP receptora deberá comprobar si se rellenó o no el control mediante la función **isset**.

En el ejemplo anterior, en la página receptora, <http://localhost/pruebas/recogida1.php> tiene este código:

```
<!DOCTYPE HTML>
<html lang="es-ES">
<head>
  <meta charset="utf-8" />
  <title></title>
</head>
<body>
  <pre>
    <?php
      print_r ($_GET);
    ?>
  </pre>
</body>
</html>
```

El resultado:

```
Array
(
    [nombre] => Jorge
    [apellidos] => Sánchez Asenjo
    [direccion] => C/ Los Vientos
    [telefono1] => 666666666
    [telefono2] => 999999999
)
```

El paso de datos de un formulario mediante GET tiene estas connotaciones:

- La URL es visible, por lo que todo lo que el usuario ha escrito está a la vista en la URL. Y eso incluye las contraseñas y nombres de usuario que haya utilizado.
- Las páginas generadas con GET pueden añadirse como marcador o favorito a los navegadores y páginas de marcadores. Eso puede ser interesante, pero puede ser un problema
- No tenemos que pasar por el formulario para enviar valores a la página receptora. Eso puede provocar problemas de seguridad, pero también puede ser una ventaja para redireccionar páginas pasando parámetros a las mismas.
- La depuración de nuestros programas es más cómoda si usamos GET, porque vemos lo que realmente llega del formulario en la propia URL.

paso de parámetros mediante POST

Los formularios se crean igual en ambos casos, pero ahora cambiamos el atributo **method** de la etiqueta **form** para elegir POST.

Ahora la URL no añade los parámetros del formulario, quedan invisibles. Realmente se pasan en el paquete http que comunica a ambas páginas, pero no se ve en la URL con lo que habría que utilizar métodos más sofisticados para poder recoger los datos que el usuario introdujo en el formulario.

(2.6.4) recepción de datos de un formulario desde una página PHP

PHP permite recibir los datos pasados por los parámetros usando dos variables, que son en realidad dos arrays que contendrán todos los valores del formulario. Se trata de **\$_GET** y de **\$_POST** cada una dedicada al método con su mismo nombre.

Puesto que son arrays (se estudian con profundidad más adelante) su manejo es distinto al de una variable normal. Podemos entender que son contenedores de todos los elementos del formulario.

Los controles del formulario que hayan recibido valores envían esos valores asociados al nombre de control especificado en el atributo **name**. De modo que si el método de paso era POST, la página PHP accederá al valor de ese control mediante:

```
$_POST["name"]
```

donde **name** es el nombre del control.

Puede ser que el control no reciba ningún valor (porque el usuario no rellena nada en él). En ese caso la invocación a **\$_POST** (o a **\$_GET**) usando el nombre de ese control, provocará un error de **variable indefinida**, por lo que debemos comprobar la existencia de la variable mediante la función **isset**.

Otras veces lo que ocurrirá es que se recibe como definida la variable pero no contiene valores. En ese caso habría que utilizar la función **empty** para comprobar que no está vacío el atributo.

(2.6.5) usar la misma página para el formulario y la recepción

A veces se usa la misma página para recibir los datos del formulario. En ese caso la página del formulario será PHP. Un uso habitual es para validar los datos desde PHP. La página recibe los datos y les valida, si está bien entonces les da por buenos sino, informa del error.

Actualmente las validaciones de los datos se realizan por JavaScript y especialmente usando AJAX; esta técnica permite validar los datos de forma más directa (sin tener que reenviarles). Pero implica usar otros lenguajes y tampoco podrían hacer validaciones complejas.

En el siguiente ejemplo se expone una página que pide un nombre de usuario y contraseña en un formulario. Si son correctos se indicará que lo son; sino, un enlace nos volvería a llevar a la página para volver a empezar. La cuestión es ¿cómo sabe la página web cuando mostrar el formulario y cuando no? La forma es valorar la existencia de la variable superglobal `$_POST` (lo mismo para `$_GET`). Cuando devuelve falso, es que no ha recogido valor alguno, es decir habría que mostrar el formulario. Código:

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title></title>
  </head>
  <body>

    <?php
      if(!$_POST) { //si no existe $_POST, el formulario se debe mostrar
        ?>

        <form action="form1.php" method="post">
          <label for="nombre">Escriba su usuario</label>
          <input type="text" name="nombre" maxlength="20" /> <br />
          <label for="contra">Escriba la contraseña</label>
          <input type="password" name="contra" maxlength="20" /> <br />
          <input type="submit" value="enviar"/><br />
        </form>

        <?php
          }//cierre del if
          else{
            $nombre=$_POST["nombre"];
            $contra=$_POST["contra"];
            if($nombre!="Jorge" || $contra!="123456") {
              echo "No es válido ese usuario y contraseÃ±a <br />";
              echo "<a href='form1.php'>Volver al formulario</a>";
            }
          }
        </?php>
      }
    </?php>
  </body>
</html>
```



```
else {  
    echo "Entrada correcta";  
}  
}  
?>  
</body>  
</html>
```

(2.7) funciones

(2.7.1) introducción

Uno de los problemas habituales del programador ocurre cuando los programas alcanzan un tamaño considerable en cuanto a líneas de código. El problema se puede volver tan complejo que acaba siendo inabordable.

Para mitigar este problema apareció la **programación modular**. En ella el programa se divide en módulos de tamaño manejable. Cada módulo realiza una función muy concreta y así el programador se concentra en cada módulo y evita la complejidad de manejar el problema completo. Los módulos se pueden programar de forma independiente. Se basa en concentrar los esfuerzos en resolver problemas sencillos y una vez resueltos, el conjunto de las soluciones a esos problemas soluciona el problema original.

En definitiva la programación modular implementa el paradigma **divide y vencerás**, tan importante en la programación. El programa se descompone en módulos. Los módulos se puede entender que son pequeños programas que reciben datos y a partir de ellos realizan un cálculo o una determinada tarea. Una vez el módulo es probado y validado se puede utilizar las veces que haga falta en el programa sin necesidad de tener que volver a programar; incluso se puede utilizar en diferentes programas ya que se pueden almacenar funciones en un mismo archivo formando lo que se conoce como **librería**.

En PHP la programación modular se implementa mediante funciones. Las funciones trabajan de esta manera:

- (1) Las funciones poseen un nombre, un identificador que cumple las reglas indicadas para los demás identificadores que conocemos (como los de las variables). Pero, a diferencia de las variables, no utilizan el signo \$. Se aconseja que el nombre de las funciones se escriba en minúsculas.
- (2) A las funciones se les indica, aunque no a todas, unos valores (parámetros) que la función necesita para hacer su labor
- (3) Las funciones pueden devolver un valor, resultado del trabajo de la misma.
- (4) Las funciones contienen el código que permite realizar la tarea para la que se creó la función.
- (5) Lo más importante es que las funciones son fundamentales en la programación de aplicaciones PHP porque permiten centrarse en un problema concreto y pequeño.

De hecho PHP incorpora muchas funciones ya creadas para trabajar (como la propia **print** o la muy utilizada **echo**). Por ejemplo este código:

```
echo rand(1,10);
```

Escribe un número aleatorio entre uno y diez. Los parámetros son el número uno y el diez, el valor que se devuelve es el número aleatorio entre esos dos números, el identificador es **rand** y al ser una función estándar de PHP, no podemos ver su código.

(2.7.2) declaración y uso de funciones personales

Los programadores de PHP pueden crear sus propias funciones para trabajar. La sintaxis es la siguiente:

```
function nombreDeLaFunción(listaDeParámetros){
    código de la función
}
```

Entre el código de la función se puede encontrar la palabra **return** que sirve para devolver el resultado.

```
function doble($valor){
    return 2*$valor;
}
```

Este código se puede declarar en cualquier parte de una página PHP, pero lo aconsejable es declarar las funciones en la cabecera.

Para utilizar una función simplemente hay que invocarla pasando los parámetros que requiere, por ejemplo:

```
$x=9.75;
echo doble(8)."<br />"; //escribe 16
echo doble($x);         //escribe 19.5
```

No siempre las funciones devuelven valores, por ejemplo:

```
function negrita($texto){
    echo "<strong>".$texto."</strong>";
}
```

La función no tiene instrucción **return**, puesto que no devuelve valores sino que lo que hace es escribir el texto en negrita. Ejemplo de uso:

```
negrita("Hola");
```

(2.7.3) alcance de las variables

Las variables definidas en una función, al finalizar la función se eliminan. Es decir su ámbito es local a la función. Ejemplo:

```
function f1(){  
    $h=9;  
}  
echo $h;
```

La instrucción **echo** del ejemplo anterior, provoca un fallo de variable no definida, porque PHP no reconoce a la variable **\$h**, la única \$h del código se crea en la función y sólo se puede usar en la función; hacer referencia a \$h fuera de la función no tiene sentido; ya que tras el cierre de la llave en la que se definió, la variable muere. La variable es, en definitiva, local a la función.

Este código:

```
$h=5;  
function f1(){  
    $h=9;  
}  
echo $h; //escribe 5
```

queda claro que una variable no se puede usar fuera de la función. Para PHP en el ejemplo anterior hay dos variables **\$h** cuando se cierra la función la que se ha definido dentro de la función desaparece y regresa la anterior que valía cinco.

(2.7.4) paso de parámetros por referencia

Por defecto las funciones reciben los parámetros por valor, esto significa que se recibe una copia del valor en la función. Por lo que examinando este código:

```
function f1($x){  
    $x=9;  
}  
$x=7;  
f1($x);  
echo $x;
```

En el ejemplo la función **f1** simplemente cambia el valor del parámetro **\$x**; al salir de esa función **\$x** muere. Por lo que al escribir lo que vale \$x, se mostrará el valor de 7, ya que el \$x de fuera de la función es diferente al parámetro.

Sin embargo a veces sí se desea que las funciones cambien el valor de las variables que se pasan como parámetro. El ejemplo más claro es el de la función **swap**. La función swap sirve para intercambiar los valores de dos valores.

Se desea que esta instrucción: `swap($x,$y)`, sirva para intercambiar los valores de `x` e `y` por ello se programa de esta forma:

```
function swap($x,$y){
    $aux=$x;
    $x=$y;
    $y=$aux;
}
$valor1=12;
$valor2=17;
swap($valor1,$valor2);
echo $valor1." ".$valor2;
```

En apariencia `swap` funciona, pero realmente no modifica los valores de las variables `$valor1` y `$valor2` puesto que los parámetros (`$x` y `$y`) reciben una copia de los valores. El problema es que esta función recibe los parámetros por valor, luego no modifica las variables originales. Por ello, `echo` escribiría `12 17`.

Cuando los parámetros se usan por referencia, entonces sí se cambian las variables. Para ello basta con preceder el signo `&` a los parámetros que se usarán por referencia. Así `swap` se programaría así:

```
function swap(&$x,&$y){
    $aux=$x;
    $x=$y;
    $y=$aux;
}
$valor1=12;
$valor2=17;
swap($valor1,$valor2);
echo $valor1." ".$valor2;
```

Escribe `17 12`, las variables originales han sido modificadas.

Cuando una función utiliza paso por referencia, entonces al invocarla debe recibir nombres de variables y no expresiones o valores literales. Es decir este código:

```
swap(9,8);
```

produce un error, porque la función espera variables y no expresiones.

(2.7.5) parámetros predefinidos

Se puede indicar un valor por defecto a los parámetros a fin de permitir que dicho parámetro sea opcional; de modo que, si se pasa el parámetro se toma el valor que se pasa y si no, se toma el valor por defecto. Ejemplo:

```
function potencia($base,$exponente=2){
    $valor=1;
    for($i=1;$i<=$exponente;$i++){
        $valor*=$base;
    }
    return $valor;
}
echo potencia(2,3); //Escribe 8, es decir 2³
echo potencia(2);  //Escribe 4, es decir 2²
```

La segunda vez que se invoca a la función, el parámetro *\$exponente* toma el valor por defecto 2.

(2.7.6) variables globales

Las variables globales permiten ser utilizadas en todo el código, sin que las funciones utilicen variables que se han definido fuera de la función. Ejemplo:

```
function encadenar($numero,$carácterRelleno){
    global $texto;
    for($i=1;$i<=$numero;$i++){
        $texto.=$carácterRelleno;
    }
}

$texto="Hola";
encadenar(12,"d");
echo $texto."<br />"; //Escribe: Holadddddddddddddd
encadenar(9,"+-");
echo $texto."<br />"; //Escribe: Holadddddddddddddd+-+--+--+--+--+--+--
```

La función encadenar utiliza la variable global *\$texto* gracias al uso de la palabra **global**. Sin embargo en realidad esto es una mala práctica ya que impide modular bien un programa ya que las funciones deberían ser lo más independientes posibles. La función del ejemplo no podría ser transportada a otra página PHP, ya que las variables globales serían otras seguramente.

En resumen, es una posibilidad que ofrece PHP pero que es poco recomendable porque con ella se adquieren malas mañas al programar.

(2.7.7) variables estáticas

Otro uso de variables especiales lo supone las variables estáticas. Se trata de variables locales a la función (sólo se podrán utilizar dentro de la función), pero son variables que recuerdan su valor entre llamadas. Ejemplo:

```
function estatica(){  
    static $cuenta=0;  
    $cuenta++;  
    echo "Esta es la llamada número $cuenta<br />";  
}  
  
for($i=1;$i<=10;$i++){  
    estatica();  
}
```

La salida del código anterior es:

```
Esta es la llamada número 1  
Esta es la llamada número 2  
Esta es la llamada número 3  
Esta es la llamada número 4  
Esta es la llamada número 5  
Esta es la llamada número 6  
Esta es la llamada número 7  
Esta es la llamada número 8  
Esta es la llamada número 9  
Esta es la llamada número 10
```

Luego en cada llamada la variable *\$cuenta* incrementa su valor. Eso sólo es posible gracias a la línea en la que se define la variable como estática (*static \$cuenta=0*) que solo se ejecuta en la primera llamada. Si en esa línea eliminamos la palabra **static**, entonces la variable se crea en cada llamada y las diez líneas anteriores escribiría lo mismo (*Esta es la llamada número 1*)

(2.7.8) recursividad

La recursividad es una técnica de creación de programas, pensada para soluciones a problemas complejos. Se basa en que dentro del código de la función se invoca a la propia función.

Esta técnica es peligrosa ya que se pueden generar fácilmente llamadas infinitas (la función se llama a sí misma, tras la llamada se vuelve a llamar a sí misma,...). Hay que ser muy cauteloso con ella (incluso evitarla si no es necesario su uso); pero permite soluciones muy originales y abre la posibilidad de solucionar problemas muy complejos. De hecho ciertos problemas serían casi imposibles de resolver sin esta técnica.

La idea general es que en cada llamada a la función, ésta resuelva parte del problema y se invoque a sí misma de nuevo para resolver la parte que queda, y así sucesivamente. En cada llamada el problema debe ser cada vez más sencillo hasta llegar a una llamada en la que la función devuelve un único valor. Tras esa llamada los resultados se encadenan hasta llegar al código que realizó la primera llamada y pasarle la solución.

Es fundamental tener en cuenta cuándo la función debe dejar de llamarse a sí misma; es decir, es importante decidir cuándo acabar. Toda función recursiva debe de tener una condición de fin de las llamadas. De otra forma se corre el riesgo de generar infinitas llamadas. Un bucle infinito utilizando recursividad puede causar graves inestabilidades en el ordenador de trabajo.

Como ejemplo vamos a ver la versión recursiva del factorial.

```
function factorial($n){  
    if($n<=1) return 1;  
    else return $n*factorial($n-1);  
}
```

La última instrucción (**return \$n*factorial(\$n-1)**) es la que realmente aplica la recursividad. La idea (por otro lado más humana) es considerar que el factorial de nueve es, nueve multiplicado por el factorial de ocho; a su vez el de ocho es ocho por el factorial de siete y así sucesivamente hasta llegar al factorial de uno, que devuelve uno.

Para la instrucción **factorial(4)**; usando el ejemplo anterior, la ejecución del programa generaría los siguientes pasos:

- (1) Se llama a la función factorial usando como parámetro el número 4 que será copiado en la variable-parámetro **n**
- (2) Como $n > 1$, entonces se devuelve 4 multiplicado por el resultado de la llamada **factorial(3)**
- (3) La llamada anterior hace que el nuevo **n** (variable distinta de la anterior) valga 3, por lo que esta llamada devolverá 3 multiplicado por el resultado de la llamada **factorial(2)**
- (4) La llamada anterior devuelve 2 multiplicado por el resultado de la llamada **factorial(1)**
- (5) Esa llamada devuelve 1
- (6) Eso hace que la llamada **factorial(2)** devuelva $2 \cdot 1$, es decir 2
- (7) Eso hace que la llamada **factorial(3)** devuelva $3 \cdot 2$, es decir 6
- (8) Por lo que la llamada **factorial(4)** devuelve $4 \cdot 6$, es decir **24** Y ese es ya el resultado final

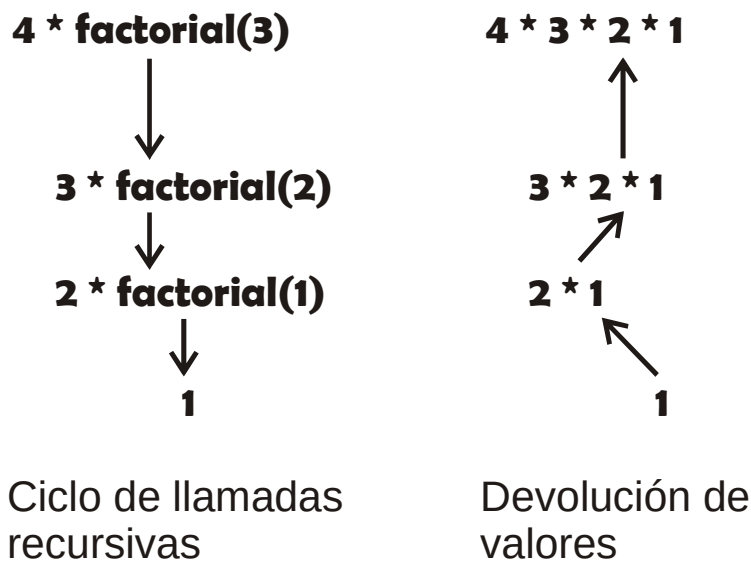


Ilustración 1, Pasos para realizar la recursividad

¿recursividad o iteración?

Hay otra versión de la función factorial resuelta mediante un bucle **for** (solución iterativa) en lugar de utilizar la recursividad. En concreto es:

```
function factorial2($n){  
    $res=1;  
    for($i=1;$i<=$n;$i++){  
        $res*=$i;  
    }  
    return $res;  
}
```

La cuestión es ¿cuál es mejor?

Ambas implican sentencias repetitivas hasta llegar a una determinada condición. Por lo que ambas pueden generar programas que no finalizan si la condición nunca se cumple. En el caso de la iteración es un contador o un centinela el que permite determinar el final, la recursividad lo que hace es ir simplificando el problema hasta generar una llamada a la función que devuelva un único valor.

Para un ordenador es más costosa la recursividad ya que implica realizar muchas llamadas a funciones en cada cual se genera una copia del código de la misma, lo que sobrecarga la memoria del ordenador. Es decir, **es más rápida y menos voluminosa la solución iterativa de un problema recursivo.**

¿Por qué elegir recursividad? De hecho si podemos resolver la función mediante la solución iterativa, no deberíamos utilizar recursividad. **La recursividad se utiliza sólo si:**

- No encontramos la solución iterativa a un problema
- El código es muchísimo más claro en su versión recursiva y no implica un gran coste de procesamiento al ordenador

(2.7.9) ámbito de las funciones

Las funciones en PHP siempre son globales. Es decir, una función se puede utilizar en cualquier parte del código PHP. Por supuesto esto prohíbe poner el mismo nombre a dos funciones.

(2.8) inclusión de ficheros

Dentro del código PHP se puede hacer uso de las instrucciones **include** y/o **require** para incluir el código de otro archivo. El archivo puede ser de cualquier tipo: **html**, **php** u otras extensiones.

Por lo que permite la creación de archivos que contengan librerías de funciones o código reutilizable en múltiples páginas.

Tanto **include** como **require** lo que hacen es simplemente copiar y pegar el código del archivo tal cual. La diferencia es que si el archivo no existe (o no se encuentra), **include** seguirá ejecutando el código (aunque normalmente mostrará una advertencia del error), mientras que **require** generará un error grave y parará la ejecución del código.

En la inclusión se entiende que el código que se incluye es código PHP, por lo que se entenderá que lo que no esté encerrado en etiquetas **<? y ?>** es código HTML.

Ejemplo de uso:

```
include("funciones.php");
```

Unidad 3

PHP (II). arrays, strings y fechas

Apuntes realizados para la asignatura de FP Grado Superior:
Implantación de Aplicaciones Web
del ciclo Administración de Sistemas Informáticos en Red

Autor: Jorge Sánchez Asenjo (www.jorgesanchez.net)
Versión del documento: 2.03, Año 2013



Esta obra está bajo una licencia de Reconocimiento-NoComercial-CompartirIgual de Creative Commons
Para ver una copia de esta licencia, visite: <http://creativecommons.org/licenses/by-nc-sa/3.0/deed.es>



Reconocimiento-NoComercial-CompartirIgual 2.5 España

Usted es libre de:



copiar, distribuir y comunicar públicamente la obra



hacer obras derivadas

Bajo las condiciones siguientes:



Reconocimiento. Debe reconocer los créditos de la obra de la manera especificada por el autor o el licenciador (pero no de una manera que sugiera que tiene su apoyo o apoyan el uso que hace de su obra).



No comercial. No puede utilizar esta obra para fines comerciales.



Compartir bajo la misma licencia. Si altera o transforma esta obra, o genera una obra derivada, sólo puede distribuir la obra generada bajo una licencia idéntica a ésta.

- Al reutilizar o distribuir la obra, tiene que dejar bien claro los términos de la licencia de esta obra.
- Alguna de estas condiciones puede no aplicarse si se obtiene el permiso del titular de los derechos de autor
- Apart from the remix rights granted under this license, nothing in this license impairs or restricts the author's moral rights.

Advertencia

Los derechos derivados de usos legítimos u otras limitaciones reconocidas por ley no se ven afectados por lo anterior.
Esto es un resumen legible por humanos del texto legal (la licencia completa) disponible en los idiomas siguientes:

Catalán Castellano Euskera Gallego

Para ver una copia completa de la licencia, acudir a la dirección
<http://creativecommons.org/licenses/by-nc-sa/2.5/es/legalcode.es>

índice

(3.1) arrays	5
(3.1.1) introducción a los arrays.....	5
(3.1.2) arrays escalares	6
(3.1.3) arrays asociativos.....	9
(3.1.4) bucle foreach	10
(3.1.5) arrays multidimensionales.....	11
(3.1.6) inspección de arrays mediante funciones de recorrido	13
(3.1.7) funciones y arrays	14
(3.1.8) funciones estándar de uso con arrays.....	14
(3.1.9) uso de arrays en formularios	22
(3.2) strings	24
(3.2.1) introducción	24
(3.2.2) asignación de valores.....	24
(3.2.3) concatenación de textos.....	24
(3.2.4) uso de variables en strings. uso de llaves	25
(3.2.5) manejo de strings como arrays de caracteres	25
(3.2.6) cadenas heredoc	25
(3.2.7) cadenas nowdoc	26
(3.2.8) expresiones regulares	26
(3.2.9) funciones estándar de uso con strings.....	31
(3.3) funciones de fecha	38

(3) PHP (II). estructuras de datos y sesiones

(3.1) arrays

(3.1.1) introducción a los arrays

Los tipos de datos que conocemos hasta ahora no permiten solucionar problemas que requieren gestionar muchos datos a la vez. Por ejemplo, imaginemos que deseamos almacenar las notas de una clase de 25 alumnos, no habrá más remedio que declarar 25 variables.

Eso es tremendamente pesado de programar. Manejar esos datos significaría estar continuamente manejando 25 variables. Por supuesto si necesitamos 2000 notas, el problema se hace inmanejable.

Por ello en casi todos los lenguajes se pueden agrupar una serie de variables del mismo tipo en una misma estructura que comúnmente se conoce como array¹. Esa estructura permite referirnos a todos los elementos, pero también nos permite acceder individualmente a cada elemento.

Los arrays son una colección de datos del mismo tipo al que se le pone un nombre (por ejemplo *nota*). Para acceder a un dato individual de la colección hay que utilizar su posición. La posición es un número entero que se llama **índice**, así para acceder a (por ejemplo *nota[4]* es el nombre que recibe el cuarto elemento de la sucesión de notas).

Hay que tener en cuenta que en los arrays el primer elemento tiene como índice el número cero. El segundo el uno y así sucesivamente; es decir *nota[4]* en realidad es el quinto elemento del array.

¹ Otros nombres habituales además de *arrays* son: listas, matrices, arreglos,... Estimo que **array** es el más aceptado en la actualidad, los otros términos son más ambiguos.

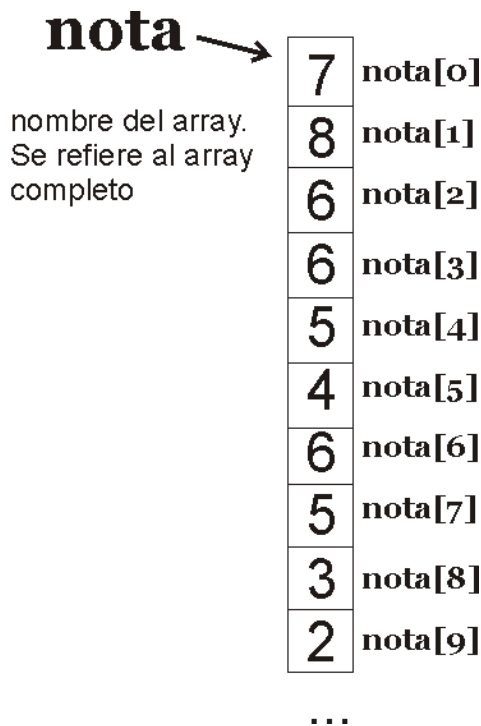


Ilustración 3-1, Ejemplo de array de notas

Esta definición de arrays es la común en todos los lenguajes clásicos. En el caso de PHP, los arrays son elementos más complejos, puesto que admiten datos de distinto tipo. La realidad es que los arrays de PHP son en realidad elementos que asocian un valor y una clave (arrays asociativos) como se explica más adelante.

(3.1.2) arrays escalares

Los arrays escalares cumplen la definición indicada anteriormente de array: es un conjunto de valores a los que se accede a través de un índice que es un número entero. Así, por ejemplo las notas se podrían almacenar de esta forma:

```
$nota[0]=5;  
$nota[1]=9;  
$nota[2]=8;  
$nota[3]=5;  
$nota[4]=6;  
$nota[5]=7;
```

Acceder a una nota cualquiera, requiere conocer su índice.

No hace falta declarar el array (siguiendo el estilo habitual de PHP), simplemente se utiliza. Es posible incluso este código:

```
$valor[0] =18;  
$valor[1]="Hola";  
$valor [2]=true;  
$valor[3]=3.4;
```

Es decir, los datos pueden ser heterogéneos. Sin indicar índice podemos:

```
$valor[] =18;  
$valor[]="Hola";  
$valor []=true;  
$valor[]=3.4;
```

Código equivalente al anterior, ya que al no indicar índice se usa el siguiente índice libre en el array.

recorrer arrays

La gracia de los arrays en todos los lenguajes es la facilidad de rellenar o utilizar su contenido mediante bucles **for**. Imaginemos que necesitamos almacenar 1000 números aleatorios del 1 al 10; evidentemente sería terrible tener que escribir 1000 líneas de código para realizar esta labor:

```
$valor[0]=rand(1,10);  
$valor[1]=rand(1,10);  
$valor[2]=rand(1,10);  
...//¡¡¡mucho trabajo!!!
```

Lo lógico es utilizar este código:

```
for($i=0;$i<1000;$i++)  
    $valor[$i]=rand(1,10);
```

La variable **\$i** sirve para recorrer cada elemento del array. Va cambiando de 0 a 999 y así en dos líneas se rellena el array.

función count

La función **count** que posee PHP permite saber el tamaño de un array. Es útil para utilizar un array del que previamente no sabemos su tamaño.

Por ejemplo si deseamos recorrer un array que ya tiene valores para, por ejemplo, mostrar cada elemento en pantalla, el código sería:

```
for($i=0;$i<count($valor);$i++){  
    echo $valor[$i]."<br />";  
}
```

elementos vacíos

En este código:

```
$nota[0]=5;  
$nota[1]=9;  
$nota[3]=5;  
$nota[4]=6;  
$nota[5]=7;
```

Deliberadamente se ha dejado sin rellenar el elemento **nota[2]**. De modo que la cuestión es ¿qué sacaría por pantalla la instrucción **echo \$nota[2]**?

Normalmente un servidor Apache en el que se ejecute este código mostraría un mensaje parecido a este:

Notice: Undefined offset: 2 in E:\Mis documentos\xampp\htdocs\arrays\arrayPrb1.php on line 18

Undefined offset se puede traducir como *desplazamiento indefinido* y además indica que dicho desplazamiento es el número **2**. Es decir, lo que Apache indica es que al moverse al elemento **2** del array se encuentra con que es una posición indefinida: dicho de otra forma, que es un índice de array que no existe.

La función **count** devuelve el tamaño del array sin tener en cuenta los elementos vacíos; es decir, sólo cuenta los elementos definidos. Por todo ello el código anterior:

```
for($i=0;$i<count($valor);$i++){  
    echo $valor[$i]."<br />";  
}
```

Puede fallar si el array *\$valor* tiene elementos vacíos. Además los últimos elementos no saldrían por que el contador no llega a ellos al contar elementos y no los índices más altos.

Para evitar eso podemos utilizar la función **isset** que devuelve verdadero cuando a la variable que se le pasa como parámetro se le asignado ya un valor. El bucle quedaría entonces:

```
$tope=count($nota);  
for($i=0;$i<$tope;$i++){  
    if(isset($nota[$i]))  
        echo $nota[$i]."<br />";  
    else  
        $tope++;  
}
```

El código es más enrevesado. Ahora sólo se escribe cada elemento del array si la función **isset** nos asegura que dicho elemento tiene valor; la variable **\$tope** va incrementando su valor a fin de alcanzar al último elemento real del array

uso del constructor **array()**

Los arrays pueden definirse de forma cómoda mediante la función **array()**. Dentro de esa función se colocan los valores que tendrá el array; y así en una sola línea podemos asignar todos los valores del array. Ejemplo:

```
$numero=array(17, 34, 45, 2, 9, -5, 7);
```

Esa línea asigna valores a cada elemento del array empezando por el cero. Es decir, es equivalente a:

```
$numero[0]=17;  
$numero[1]=34;  
$numero[2]=45;  
$numero[3]=2;
```

```
$numero[4]=9;  
$numero[5]=-5;  
$numero[6]=7;
```

Esta función tiene posibilidad de elegir índices concretos. Eso se hace colocando el índice y después el símbolo **=>** antes del valor. Ejemplo:

```
$numero=array(17, 34, 3=>45, 2, 7=>9, 10=>-5, 7);
```

Sería equivalente a:

```
$numero[0]=17;  
$numero[1]=34;  
$numero[3]=45;  
$numero[4]=2;  
$numero[7]=9;  
$numero[10]=-5;  
$numero[11]=7;
```

Los índices que no se indican siguen la serie natural

(3.1.3) arrays asociativos

En este caso el índice es un texto que se comporta como una especie de clave que permite acceder al dato. Ejemplo:

```
$nota["Antonio"]=5;  
$nota["Luis"]=9;  
$nota["Ana"]=8;  
$nota["Eloy"]=5;  
$nota["Gabriela"]=6;  
$nota["Berta"]=7;
```

La ventaja de estos arrays es que son más legibles, la desventaja es que no permiten su manejo mediante los bucles clásicos.

Esos arrays también permiten el uso de la función **array()**:

```
$nota=array("Antonio"=>5,"Luis"=>9,"Ana"=>8,"Eloy"=>5,  
"Gabriela"=>6,"Berta"=>7);
```

Esta función incluso permite mezclar valores escalares y asociativos (aunque no es muy recomendable), por ejemplo:

```
$nota=array("Antonio"=>5, 4, 7, 5=>6, "Luis"=>9, 2,"Berta"=>7);
```

El ejemplo anterior sería equivalente a:

```
$nota["Antonio"]=5;
$nota[0]=4;
$nota[1]=7;
$nota[5]=6;
$nota["Luis"]=9;
$nota[6]=2;
$nota["Berta"]=7;
```

Otra cosa es la utilidad de esa sintaxis, que al final complica en demasía el código.

(3.1.4) bucle foreach

Recorrer arrays asociativos o incluso escalar con elementos si inicializar resulta muy complejo con los bucles **while** o **for**. Por ello PHP dispone de un bucle muy potente que permite recorrer todos los elementos de un array sin importar si hay saltos en la numeración o si se trata de un array asociativo. Se trata del bucle **foreach** que tiene esta sintaxis:

```
foreach(array as índice=>valor ){
    sentencias
}
```

array es el nombre del array que se va a recorrer; **índice** es la variable que recogerá el **índice** de cada elemento a medida que se recorra el array; y **valor** es la variable que irá recogiendo el valor de cada elemento del array. Ejemplo:

```
$nota=array("Antonio"=>5, 4, 7, 5=>6, "Luis"=>9, 2,"Berta"=>7);
foreach ($nota as $i=>$v){
    echo "La nota $i vale $v\n";
}
/* Escribe:
    La nota Antonio vale 5
    La nota 0 vale 4
    La nota 5 vale 6
    La nota Luis vale 9
    La nota 6 vale 2
    La nota Berta vale 7
*/
```

(3.1.5) arrays multidimensionales

Un array multidimensional sirve para representar datos agrupados en una estructura que utiliza dos índices para representar cada elemento del array. Ejemplos:

```
$nota[0][0]=9;  
$nota[0][1]=7;  
$nota[1][0]=9;  
...
```

Por supuesto es posible utilizar cualquier tipo de valor tanto para los índices como para el valor:

```
$poblacion["Alemania"]["Berlin"]=4000000;  
$poblacion["Alemania"]["Hanover"]=1200000;  
$poblacion["Francia"]["Paris"]=7400000;  
$poblacion["Francia"]["Lyon"]=2300000;  
$poblacion["España"]["Palencia"]=80000;
```

Y se pueden utilizar más de dos dimensiones:

```
$poblacion["España"]["Castilla y León"]["Palencia"]=80000;  
$poblacion["España"]["Castilla y León"]["Valladolid"]=370000;  
$poblacion["España"]["Asturias"]["Oviedo"]=115000;  
$poblacion["Alemania"]["Brandemburgo"]["Berlin"]=4000000;
```

De esa forma podemos considerar a los arrays como una especie de almacenes de datos que se parecen mucho a las tablas de las bases de datos relacionales; ese detalle facilita mucho la comunicación entre PHP y las bases de datos.

Otra forma posible:

```
$alemania=array("Berlin", "Hannover");  
$francia=array("París", "Lyon");  
$espania=array("Palencia");  
$poblacion=array("alemania"=>$alemania,"francia"=>$francia,  
"espania"=>$espania);
```

constructor **array()** en arrays multidimensionales

La función **array()** está pensada para arrays de una dimensión. Pero es posible utilizarla en arrays multidimensionales considerando que un array de dos dimensiones es un array de arrays. De esa forma:

```
$poblacion=array("España"=>array("Palencia"=>80000,"Valladolid"=>350000,  
"Oviedo"=>120000),  
"Francia"=>array("Paris"=>7000000,"Lyon"=>2100000)  
);
```

Es equivalente a:

```
$poblacion["España"]["Palencia"]=80000;  
$poblacion["España"]["Valladolid"]=350000;  
$poblacion["España"]["Oviedo"]=120000;  
$poblacion["Francia"]["Paris"]=7000000;  
$poblacion["Francia"]["Lyon"]=2100000;
```

recorrido mediante **foreach** de arrays multidimensionales

Nuevamente se basa la idea en considerar que un array multidimensional es un array de arrays. Así en el que caso de que con el array anterior hiciéramos este recorrido:

```
$poblacion=array("España"=>array("Palencia"=>80000,"Valladolid"=>350000,  
                                "Oviedo"=>120000),  
                "Francia"=>array("Paris"=>7000000,"Lyon"=>2100000)  
                );  
foreach($poblacion as $i=$valor){  
    echo "El índice $i vale $valor<br />";  
}
```

El resultado es:

```
El índice España vale Array  
El índice Francia vale Array
```

Lo que nos indica el resultado es que el valor de cada elemento es un array, con lo cual el código que realmente nos permite examinar cada valor sería:

```
$poblacion=array("España"=>array("Palencia"=>80000,"Valladolid"=>350000,  
                                "Oviedo"=>120000),  
                "Francia"=>array("Paris"=>7000000,"Lyon"=>2100000)  
                );  
foreach ($poblacion as $pais => $ciudades) {  
    foreach ($ciudades as $ciudad => $valor) {  
        echo "<p>País: $pais, Ciudad: $ciudad, población = $valor</p>";  
    }  
}
```

Que obtiene el resultado:

```
País: España, Ciudad: Palencia, población = 80000  
País: España, Ciudad: Valladolid, población = 350000  
País: España, Ciudad: Oviedo, población = 120000  
País: Francia, Ciudad: Paris, población = 7000000  
País: Francia, Ciudad: Lyon, población = 2100000
```

(3.1.6) inspección de arrays mediante funciones de recorrido

Se trata de funciones basadas en el manejo de punteros de los lenguajes tradicionales. Utilizan un puntero virtual, que sería un objeto que señala a uno de los elementos del array y que al moverle permite acceder al resto de elementos.

Esta técnica se basa en el manejo de las siguientes funciones:

función	uso
current(array)	Devuelve el valor del elemento al que actualmente señala el puntero. Si no hay ningún elemento, devuelve false .
key(array)	Devuelve la clave a la que señala el puntero
next(array)	Mueve el puntero al siguiente elemento del array y devuelve su valor. Si el elemento actual es el último, next devuelve false .
prev(array)	Mueve el puntero al elemento anterior del array y devuelve su valor. Si el elemento actual es el primero, prev devuelve false .
reset(array)	Coloca el puntero en el primer elemento del array y devuelve su valor. Si no hay ningún elemento en el array, devuelve false .
end(array)	Coloca el puntero en el último elemento y devuelve su valor.

Ejemplo de recorrido de un array mediante estas funciones:

```
$capital=array( "Castilla y León"=>"Valladolid", "Asturias"=>"Oviedo",
               "Aragón"=>"Zaragoza");
while(current($capital)){
    echo "<strong>".current($capital)."</strong><br />";
    next($capital);
}
```

Para recorrerle al revés:

```
$capital=array( "Castilla y León"=>"Valladolid", "Asturias"=>"Oviedo",
               "Aragón"=>"Zaragoza");
end($capital);
while(current($capital)){
    echo "<strong>".current($capital)."</strong><br />";
    prev($capital);
}
```

(3.1.7) funciones y arrays

Las funciones pueden recibir y devolver arrays al igual que cualquier otro tipo de variables.

A diferencia de la mayoría de lenguajes estructurados, en PHP los arrays se pasan por valor a las funciones. Es decir cuando se pasa un array a una función, ésta recibe una copia del mismo. Así las modificaciones al array que se hagan dentro de la función, no afectan al array original. Ejemplo:

```
function prueba($a){  
    $a[0]=18;  
}  
  
$array=array(1,2,3,4,5,6,7);  
prueba($array);  
echo $array[0];
```

La función **prueba** modifica el elemento con índice cero en el array para darle el valor 18. Sin embargo al ejecutar el código PHP anterior, veremos el número **1** en pantalla. La modificación de la función prueba se ha hecho con el array **\$a** que es una copia en realidad del original **\$array**.

Si deseáramos que realmente la función modifique el array, necesitamos pasar el array por referencia, como ya se ha visto en PHP basta con indicar el símbolo & delante del parámetro (o parámetros) que deseamos pasar por referencia:

```
function prueba(&$a){  
    $a[0]=18;  
}  
  
$a=array(1,2,3,4,5,6,7);  
prueba($a);  
echo $a[0];
```

Ahora sí escribe **18**.

(3.1.8) funciones estándar de uso con arrays

funciones básicas

función	significado
count(\$array)	número de elementos del array
print_r(\$array)	Escribe el contenido del array (tanto valores como índices)
is_array(array)	Devuelve verdadero si el parámetro que recibe es un array

ordenación de arrays

función	significado
sort(array, [flags])	Ordena el array indicado. Si no se indica el segundo parámetro, ordena de forma normal sin convertir los tipos de datos. El segundo parámetro permite establecer la forma de ordenación pudiendo indicar estas posibilidades: • SORT_REGULAR. Ordenación normal (utilizada por defecto), se adapta según el valor sea numérico o String y no usa ordenación basado en alfabetos distintos del inglés. • SORT_NUMERIC. Ordena entendiendo que el array contiene valores numéricos • SORT_STRING. Ordena entendiendo que el array contiene valores String (texto) • SORT_LOCALE_STRING. Ordena basándose en la configuración regional establecida con la función setlocale. De modo que setlocale(LC_ALL,"es_ES") establecería como configuración regional, español de España. • SORT_NATURAL. Ordena el array considerando sus valores como lo haría un ser humano (es decir el texto "texto2" iría delante de "texto10" porque entendería el diez como un número superior al dos, y no ordenaría como si fueran textos alfabéticos. El funcionamiento es el mismo que la función natsort • SORT_FLAG_CASE Permite combinar (usando el operador OR () a nivel de bits) con SORT_STRING o SORT_NATURAL para ordenar cadenas de forma insensible a mayúsculas/minúsculas. Por ejemplo: <code>sort(\$array,SORT_STRING SORT_NATURAL)</code> Al ordenar de esta forma, los índices desaparecen, el array final es un array escalar (el primer índice será el cero, luego el uno, etc.)
rsort(array, [flags])	Igual que la anterior, pero ordena en orden descendente.
asort(array, [flags])	Idéntica a sort , pero los índices no se desprecian y se respetan. Es decir se ordenan los valores y los índices se recolocan junto a la posición de su valor correspondiente
arsort(array, [flags])	Como la anterior pero el orden es descendente
krsort(array, [flags])	Idéntica a sort , pero se ordenan las claves. La relación de la clave junto con su valor correspondiente se respeta.
krsort(array, [flags])	Como la anterior pero el orden es descendente

función	significado
usort(array, funcionUsuario)	Ordena el array utilizando una función de usuario. Dicha función se debe crear de forma que reciba dos parámetros y devuelva un número mayor de cero cuando el primer parámetro sea mayor que cero, cero cuando sean iguales y un número menor que cero cuando el segundo parámetro sea mayor que el primero. usort recibe el nombre de la función entre comillas. Ejemplo: <pre>usort(\$array,"funcion1")</pre> Se supone que la funcion1 ha sido creada previamente y cumple los requisitos indicados anteriormente Esto permite crear ordenaciones personales con los arrays. Las claves se pierden (al igual que ocurre con la función sort) y sólo se ordenan los valores, resultando un array escalar.
uasort(array, funcionUsuario)	Igual que la anterior, pero se respeta la relación de las claves con sus valores.
uksort(array, funcionUsuario)	Igual que la anterior, pero lo que ordena son las claves en lugar de los valores.
array_multisort(array1, arg1 ó array2,...)	Permite ordenar varios arrays al mismo tiempo, indicando cada array y (opcionalmente) la forma de ordenar. La forma de ordenar se indica mediante las palabras SORT_ASC (orden ascendente), SORT_DESC (orden descendente), SORT_REGULAR (orden normal), SORT_NUMERIC (ordenación numérica), SORT_STRING (ordenación para texto). Ejemplo: <pre>array_multisort(\$a,SORT_DESC, SORT_NUMERIC, \$b,SORT_STRING)</pre> En el ejemplo, el primer array (\$a) se ordena en descendente y de forma numérica, mientras que el segundo (\$b) se ordena de forma textual.
natsort(array)	Ordena el array de modo que se ordena en la forma en la que lo haría un ser humano. Es decir el texto imagen2 iría antes que imagen10. Se respeta la relación entre clave y valor, por lo que no se pierden las claves.
natcasesort(array)	Igual que la anterior, pero no distingue entre mayúsculas y minúsculas.

búsquedas y filtros en los arrays

función	significado
<code>array_search(valorBusq,array [,estricto])</code>	Busca el valor indicado en el array y devuelve la clave del valor en el array o false si no lo encuentra. El tercer parámetro (<i>stricto</i>) es opcional y en caso de valer verdadero (por defecto vale falso), sólo encuentra el valor si tiene el mismo valor en el array y además es del mismo tipo.
<code>in_array(valorBusq,array [,estricto])</code>	Busca el valor indicado en el array, de forma estricta o no (ver función anterior) y devuelve true si lo encuentra o false en caso contrario.
<code>array_key_exists(array, clave)</code>	Devuelve verdadero si en el array existe la clave indicada.
<code>preg_prep(expresionRegular,array[,flag])</code>	Devuelve un array que contiene los valores del array que cumplen la expresión regular. En el apartado (3.2.8), página 26, se explican las expresiones regulares. Si se usa el tercer parámetro con la constante PREG_GREP_INVERT , el array resultado contiene los elementos que no cumplen la expresión regular.
<code>array_filter(array,función)</code>	El segundo parámetro es el nombre de una función existente indicada entre comillas. Dicha función se creará de modo que tenga un solo parámetro y devuelva verdadero o falso en base a una condición que se valorará en el parámetro. array_filter aplica dicha función a cada elemento del array y devuelve un nuevo array que contiene los elementos del primer array a los que, pasándolos como parámetro a la función, ésta devuelva falso. Es decir elimina todos los elementos que no cumplan la condición establecida por la función.

funciones para manejo aleatorio

función	significado
<code>shuffle(array)</code>	Reordena de forma aleatoria el array
<code>array_rand(array)</code>	Devuelve un índice aleatorio del array

funciones de manipulación del contenido del array

función	significado
<code>array_keys(\$array)</code>	Devuelve un array escalar que contiene todas las claves del array
<code>array_values(\$array)</code>	Devuelve un array escalar en el que están los valores del array (los índices se retiran y se cambian por escalares)
<code>array_flip(\$array)</code>	Intercambia las claves por los valores en el array. Es decir: las claves pasan a ser valores y los valores las claves de dichos valores.

función	significado
<code>array_combine(array1, array2)</code>	Toma los valores de ambos arrays y devuelve un nuevo array donde las claves son los valores del primer array y los valores son los del segundo array. Ambos arrays deben de tener el mismo número de valores (de otro modo ocurrirá un error). Las claves de ambos arrays se ignoran en el resultado final.
<code>array_fill(inicio,n,valor)</code>	Devuelve un array que contiene <i>n</i> elementos, todos ellos con el valor indicado. El parámetro <i>inicio</i> indica cuál será el primer índice del array (es numérico); los siguientes índices serán los números consecutivos al inicial.
<code>array_fill_keys(claves,valor)</code>	Devuelve un array a partir de otro (parámetro <i>claves</i>) array cuyos valores se considerarán las claves del nuevo. Todos los elementos del nuevo array valdrán el valor indicado.
<code>array_unique(array, [flags])</code>	Elimina los valores duplicados en el array. El segundo parámetro sirve para indicar la forma de comparar los valores y utiliza las mismas posibilidades que las indicadas en la función sort.
<code>array_pad(array,n,valor)</code>	Devuelve un nuevo array copia del primero, pero en el que se rellena del valor indicado hasta alcanzar el tamaño indicado por <i>n</i>. Si <i>n</i> es un número positivo el relleno se hace hacia la derecha y si no, hacia la izquierda. Ejemplo: <pre>\$a=array("a","b","c"); \$b=array_pad(\$a, 5, "x")</pre> El array b será: <i>a, b, c, x, x</i>
<code>array_slice(array,posInicio[,posFin])</code>	Devuelve una porción del array que se indica de modo que se toman los elementos desde la posición inicial indicada, hasta la posición final indicada. Si el parámetro <i>posFin</i> se omite, entonces se toma desde la posición inicial hasta el último elemento del array. Las posiciones indicadas se toman de forma escalar, es decir el primer elemento tiene la posición cero, el segundo uno,... El array original no cambia, pero en el resultante los índices nuevos será escalares; es decir, comienzan a numerar los índices desde el cero.

función	significado
<code>array_splice(</code> <code>array,posInicio[,tamaño</code> <code>[,arraySubst]]</code> <code>)</code>	Elimina los elementos del array usado como primer parámetro de la función desde la posición marcada mediante el parámetro posInicio hasta el final. Este parámetro puede ser negativo y entonces la posición (posInicio) desde la que eliminar se cuenta desde el final. A la hora de indicar la posInicio, se maneja el array como si fuera escalar. Es decir el primer elemento es el cero, el segundo es el uno,... Devuelve un array con los elementos eliminados. Si se usa el parámetro tamaño entonces se recorta el número indicado por ese parámetro (en lugar de llegar hasta el final). El parámetro arraySubst sirve para indicar un array que contiene los elementos con los que se sustituirán los eliminados.
<code>array_count_values(array)</code>	Genera un nuevo array en el que los índices son los valores anteriores y el valor es el número de veces que cada valor antiguo aparecía en el array original
<code>array_sum(array)</code>	Suma todos los valores del array y retorna el resultado
<code>array_product(array)</code>	Multiplica todos los valores del array y retorna el resultado
<code>array_shift(array)</code>	Retira el primer elemento del array, desplazando a todos los demás elementos en el mismo. Es decir, no deja el hueco del eliminado. Los índices se ponen de forma escalar, es decir comienzan a numerarse desde el número cero.
<code>array_unshift(array,valor1,valor2,...)</code>	Coloca al principio del array los valores indicados
<code>array_pop(array)</code>	Retira del array a su elemento y lo devuelve como resultado
<code>array_push(array,valor1, valor2,...)</code>	Coloca los valores indicados (al menos uno) al final del array original. Combinado con array_pop permite simular pilas
<code>array_map(función, array)</code>	Indica una función existente (pero entrecomillada) y devuelve un array resultado de aplicar la función indicada a cada elemento del array. La función se debe definir usando un solo parámetro, ese parámetro representa a cada elemento del array.

función	significado
<code>array_walk(array, función)</code>	Indica una función existente (pero entrecomillada) y la aplica a cada elemento del array. La función al definirla usará un solo parámetro que representa a cada elemento del array. Dicho elemento se define por referencia al crear la función (es decir utiliza el operador &) de ese modo la función realmente podrá modificar el valor de cada elemento.. Ejemplo: <pre>function doble(&\$x){ \$x*=2; } \$array=array(1,2,3,4,5); array_walk(\$array,"doble"); print_r(\$array);</pre> Escribiría: 2,4,6,8,10
<code>array_walk_recursive(array, función)</code>	Indica una función existente (pero entrecomillada) y la aplica a cada elemento del array. La función al definirla usará un solo parámetro que representa a cada elemento del array. En el caso de que los valores del array sean otros arrays, se ejecutará la función para dichos arrays.
<code>array_chunk(array,tamaño [,respetarClaves])</code>	Devuelve un array multidimensional, resultado de dividir el array que se pasa como parámetro en trozos del tamaño indicado. El parámetro opcional respetarClaves, si vale true (por defecto es false) respeta las claves originales; de otro modo las claves se pierden y cada array se renumera de forma escalar.

combinación de arrays

función	significado
<code>array_merge(\$array1,\$array2,...)</code>	Une los dos arrays que se le pasan los valores del segundo van a continuación de los del primero. Si el array es asociativo, en caso de repetir claves, sólo se queda con las últimas. <pre>\$a=array("uno"=>"Pedro","dos"=>"Antonio","tres"=>"Santiago"); \$b=array("dos"=>"Sara","tres"=>"Antonio","cuatro"=>"Santiago"); \$c=array_merge(\$a,\$b); print_r(\$c); /* obtiene: Array ([uno] => Pedro [dos] => Sara [tres] => Antonio [cuatro] => Santiago)</pre> Si el array es escalar se renumeran de nuevo todas las claves y sí aparecen las claves con índices repetidos.
<code>array_merge_recursive(\$a1,\$a2,...)</code>	Igual que el anterior, sólo que ahora si hay índices repetidos, en el array resultante se convertirán en un array que combina todos los valores de ese índice.
<code>array_intersect(\$array1,\$array2,...)</code>	Genera un nuevo array, intersección de los indicados. En el array resultante sólo aparecen los valores duplicados en todos los arrays. Se mantienen los índices, pero toma los primeros índices que parezcan en la lista de arrays.
<code>array_intersect_key(\$array1,\$array2,...)</code>	Genera un nuevo array en el que aparecen las claves presentes en todos los arrays indicados como parámetros.
<code>array_diff(\$array1,\$array2,...)</code>	Genera un nuevo array en el que aparecen los valores del primer array que no están en el segundo (array de diferencia). La relación entre clave y valor se mantiene.
<code>array_diff_key(\$array1,\$array2,...)</code>	Genera un nuevo array en el que aparecen las claves del primer array que no están en el segundo. La relación entre clave y valor se mantiene.

conversión de matrices a partir de variables comunes y viceversa

función	significado
compact (lista de variables)	Se le pasa una lista de variables. La lista es en realidad una lista de textos (strings) que contienen el nombre de las variables. La función devuelve un array con esos valores
list (listaDeVariables)=array	list no es una función, realmente permite asociar valores de un array a una lista de variables. Ejemplo: \$a=array(979797979,"Manuel",1250.45); list(\$tlfno,\$nombre,\$salario)=\$a; Las variables <i>\$tlfno</i> , <i>\$nombre</i> y <i>\$salario</i> tomarán cada una el valor que les corresponda del array.

(3.1.9) uso de arrays en formularios

Una de las virtudes de un array, es su capacidad de recoger en una sola variable, valores procedentes de diferentes controles en un formulario. La forma es muy sencilla, basta con indicar un nombre de array como atributo **name** en los elementos del formulario; para indicar que ese nombre es de un array, se añaden la apertura y cierre de corchetes [y]. Ejemplo:

```
<!DOCTYPE html>
<html lang="es-ES">
  <head>
    <meta charset="UTF-8">
    <title></title>
  </head>
  <body>
    <form action="formArrayget.php" method="GET">
      Elige estas opciones:<br />
      <input type="checkbox" name="opciones[]" value="menor" />
        Menor de edad <br />
      <input type="checkbox" name="opciones[]" value="minusvalia" />
        Minusvalía<br />
      <input type="checkbox" name="opciones[]" value="numerosa" />
        Familia numerosa <br />
      <input type="checkbox" name="opciones[]" value="minima" />
        Renta mínima<br />
      <input type="submit" value="Enviar" />
    </form>
  </body>
</html>
```

La página resultante es (suponiendo que hemos marcado después algunas opciones):

Elige estas opciones:

- ☒ Menor de edad
- ☐ Minusvalía
- ☒ Familia numerosa
- ☐ Renta mínima

En esta página, todos los checkbox están asociados a un array llamado *opciones*. Dicho array contendrá un valor por cada elemento al que le hayamos hecho clic. Los índices se indican de forma escalar, es decir el primer valor (en el orden de escritura de la página web) será el cero, luego el uno,...

De modo que si el código de la página que recoge el array (*formArrayget.php*) es:

```
<!DOCTYPE html>
<html lang="es-ES">
  <head>
    <meta charset="UTF-8">
    <title></title>
  </head>
  <body>
    <?php
      print_r($_GET["opciones"]);
    ?>
  </body>
</html>
```

La salida de dicha página tras recoger los valores del formulario anteriormente comentados es:

```
Array ( [0] => menor [1] => numerosa )
```

Indicando que el array *opciones* (presente en el array de recogida del formulario *\$_GET*) tiene dos elementos (con índices 0 y 1) con las valores *menor* y *numerosa* (correspondientes a los valores de los controles checkbox del formulario).

(3.2) strings

(3.2.1) introducción

La palabra String en inglés significa *cadena* y por eso es muy habitual en muchos libros de informática referirse así a este tipo de dato. Sin embargo en realidad por String se entiende a una serie de caracteres, lo que normalmente llamamos simplemente texto.

¿Por qué no llamarlas simplemente textos? Porque no todos los strings son textos. Es decir, hay series de caracteres que no representan necesariamente textos. Esto por ejemplo: `!"·$%&/()=?` es un String en cuanto a qué es una serie de caracteres, pero no diríamos que es un texto al no ser entendible.

En toda aplicación informática los strings son el tipo de dato fundamental. Tanto es así que en realidad ya hemos usado strings en los apartados anteriores. Sin embargo en este punto vamos a desglosar todo el manejo de strings que permite PHP.

(3.2.2) asignación de valores

Para asignar valores a un **string** basta con utilizar el operador `=`. A continuación se indica el texto entre comillas dobles o simples:

```
$string1="Soy un texto";  
$string2='Yo también';
```

En el caso de los caracteres especiales, éstos sólo se pueden indicar dentro de comillas dobles.

(3.2.3) concatenación de textos

Como se ha comentado en apartados anteriores, el operador de concatenación de textos es el símbolo del punto (`.`).

Ejemplos:

```
$texto1="Hola";  
$texto2="a todos y todas";  
$texto3=$texto1." ".$texto2;  
echo $texto3;
```

Por pantalla saldría *Hola a todos y todas*. Como se observa en el ejemplo se pueden concatenar tanto variables de tipo strings como textos literales (se encadena el contenido de la variable *\$texto* se añade un espacio en blanco y se concatena con el contenido de la variable *texto2*).

Podemos usar la concatenación y asignación:

```
$texto1="Hola ";  
$texto1.= "a todos y todas";  
echo $texto1;
```

El resultado es como en el texto anterior ya que el símbolo `.=` permite añadir al final de la variable el String que se indique.

(3.2.4) uso de variables en strings. uso de llaves

Una de las grandes capacidades PHP es la de poder traducir contenidos de variables por su valor dentro de un texto. Ejemplo:

```
$libre="free";  
echo "En inglés libre se dice $libre"; //muestra: En inglés libre se dice free
```

Sin embargo, este código daría error:

```
echo "<br />Me gusta patinar en $librestyle";
```

ya que no existe la variable *\$librestyle* (ya que no entiende que haya que considerar *\$libre* y luego *style*).

Sin embargo sí funciona mediante:

```
echo "<br />Me gusta patinar en {$libre}style";  
//Sale: Me gusta patinar en freestyle
```

las llaves son además imprescindibles para utilizar con arrays multidimensionales; aunque, en cualquier caso el operador de concatenación de texto puede ayudarnos con expresiones más complejas.

```
echo "El doble de x es".x*2."<br />";
```

(3.2.5) manejo de strings como arrays de caracteres

Se puede..... Ejemplo:

```
$string1="Este es el texto de prueba";  
$string1[6]="X";  
echo $string1; //Sale: Este eX el texto de prueba
```

Aunque realmente no se considera un array; de hecho no se admite el uso de foreach en strings.

(3.2.6) cadenas heredoc

Permiten asignar valores de texto sin usar comillas. Ejemplo:

```
$nombre="Jorge";  
$texto=<<<fin  
Mi querida amiga <br />  
escribo estas líneas esperando que me leas. <br />  
Firmado: $nombre <br />  
fin;  
echo $texto;
```

En el ejemplo se ha coloreado de color rojo el texto que se almacena en la variable *\$texto*. El texto a asignar es el que sigue al símbolo de inserción de documento (<<<) y

al **marcador de texto**, que es un grupo de caracteres concreto (en este caso se ha usado la palabra *fin* como marcador de texto). El marcador se indica inmediatamente después del símbolo <<< y vuelve a aparecer en la primera columna tras el último carácter que se almacenará en la variable. Es decir, escribe:

```
    Mi querida amiga
    escribo estas líneas esperando que me leas.
    Firmado: Jorge
```

(3.2.7) cadenas nowdoc

Funciona igual que el anterior, sólo que entiende que el texto está encerrado entre comillas, por lo que no se interpretan los nombres de variable ni los códigos de escape. Para diferenciar las dos notaciones, en ésta el marcador de línea va entre comillas simples (en la declaración, no en el cierre):

```
$nombre="Jorge";
$texto=<<<'fin'
Mi querida amiga <br />
escribo estas líneas esperando que me leas. <br />
Firmado: $nombre <br />
fin;
echo $texto;
```

Resultado:

```
    Mi querida amiga
    escribo estas líneas esperando que me leas.
    Firmado: $nombre
```

No se ha interpretado la variable *\$nombre*, se ha entendido que es un texto normal.

(3.2.8) expresiones regulares

Uno de los usos más habituales en la mayoría de lenguajes de programación, tiene que ver con expresiones regulares.

Se utilizan, sobre todo, para establecer un patrón que sirva para conseguir que ciertos textos le cumplan. Ese patrón permite búsquedas avanzadas, criterios avanzados de verificación de claves o códigos (por ejemplo números de serie de productos que cumplen unas reglas muy concretas), etc.

PHP permite el uso de dos tipos de expresiones regulares:

- **POSIX (Portable Operating System Interface) 1003.2** correspondientes a un estándar aceptado por el organismo IEEE (*The Institute of Electrical and Electronics Engineers*) muy influyente en normas electrónicas y que se basa en la sintaxis del sistema **Unix**. Las funciones compatibles con este formato comienzan por la palabra **ereg**.
- **PCRE (Perl Compatible Regular Expressions)**. Parte de PHP desde la versión 4.2, procedente del lenguaje **Perl** (precisamente famoso por su uso de las expresiones regulares). Es el formato que más se usa, de hecho se recomienda no utilizar el anterior. Las funciones que utilizan este formato de expresión regulares comienzan por la palabra **preg**. Por otro lado este formato es

compatible con Unicode, por lo que también es más recomendable para lenguas que usan símbolos fuera del ASCII original como el castellano.

formato de las expresiones regulares PCRE

Las expresiones regulares utilizan símbolos especiales para indicar el patrón correspondiente. Las expresiones regulares de tipo Perl deben ir delimitadas por la barra de dividir /. Los símbolos y patrones que se pueden utilizar son:

símbolo	significado
c	Si c es un carácter cualquiera (por ejemplo a , H , ñ , <i>etc.</i>) indica, donde aparezca dentro de la expresión, que en esa posición debe aparecer dicho carácter para que la expresión sea válida.
cde	Siendo c , d , y e caracteres, indica que esos caracteres deben aparecer de esa manera en la expresión.
(x)	Permite indicar un subpatrón dentro del paréntesis. Ayuda a formar expresiones regulares complejas.
.	Cualquier carácter. El punto indica que en esa posición puede ir cualquier carácter
^x	Comenzar por. Indica el String debe empezar por la expresión x .
x\$	Finalizar por. Indica que el String debe terminar con la expresión x .
x+	La expresión a la izquierda de este símbolo se puede repetir una o más veces
x*	la expresión a la izquierda de este símbolo se puede repetir cero o más veces
x?	El carácter a la izquierda de este símbolo se puede repetir cero o una veces
x{n}	Significa que la expresión x aparecerá n veces, siendo n un número entero positivo.
x{n,}	Significa que la expresión x aparecerá n o más veces
x{m,n}	Significa que la expresión x aparecerá de m a n veces.
x y	La barra indica que las expresiones x e y son opcionales, se puede cumplir una u otra.
c-d	Cumplen esa expresión los caracteres que, en orden ASCII, vayan del carácter c al carácter d . Por ejemplo a-z representa todas las letras minúsculas del alfabeto inglés.
[cde]	Opción, son válidos uno de estos caracteres: c , d ó e
[^x]	No es válido ninguno de los caracteres que cumplan la expresión x . Por ejemplo [^dft] indica que no son válidos los caracteres d , f ó t .
\d	Dígito, vale cualquier dígito numérico
\D	Todo menos dígito
\s	Espacio en blanco

símbolo	significado																																																														
\S	Cualquier carácter salvo el espacio en blanco																																																														
\w	Word , carácter válido dentro de los que PHP considera para identificar variables. Es decir, letras, números o el guion bajo.																																																														
\W	Todo lo que no sea un carácter de tipo Word .																																																														
\n	Nueva línea																																																														
\t	Tabulador																																																														
\c	Permite representar el carácter c cuando este sea un carácter que de otra manera no sea representable (como [,] , / , \ ,...). Por ejemplo \\ es la forma de representar la propia barra invertida.																																																														
\xff	Permite indicar un carácter Unicode mediante su código hexadecimal.																																																														
\p{xx}	Indica un carácter que cumpla la propiedad Unicode indicada con los símbolos xx que pueden ser: <table border="1"> <thead> <tr> <th>Código</th><th>Significado</th></tr> </thead> <tbody> <tr><td>Cc</td><td>Control</td></tr> <tr><td>Cf</td><td>Formato</td></tr> <tr><td>Cn</td><td>Sin asignar</td></tr> <tr><td>Co</td><td>Uso privado</td></tr> <tr><td>Cs</td><td>Sustituto</td></tr> <tr><td>L</td><td>Letra</td></tr> <tr><td>Li</td><td>Letra minúscula</td></tr> <tr><td>Lm</td><td>Letra modificadora</td></tr> <tr><td>Lo</td><td>Otra letra</td></tr> <tr><td>Lt</td><td>Letra de título</td></tr> <tr><td>Lu</td><td>Letra mayúscula</td></tr> <tr><td>M</td><td>Marca</td></tr> <tr><td>Mc</td><td>Marca de espacio</td></tr> <tr><td>Me</td><td>Marca de cierre</td></tr> <tr><td>Mn</td><td>Marca de no-espacio</td></tr> <tr><td>N</td><td>Número</td></tr> <tr><td>Nd</td><td>Número decimal</td></tr> <tr><td>NI</td><td>Número letra</td></tr> <tr><td>No</td><td>Otro número</td></tr> <tr><td>P</td><td>Puntuación</td></tr> <tr><td>Pc</td><td>Puntuación de conexión</td></tr> <tr><td>Pd</td><td>Puntuación guión</td></tr> <tr><td>Pe</td><td>Puntuación de cierre</td></tr> <tr><td>Pf</td><td>Puntuación final</td></tr> <tr><td>Pi</td><td>Puntuación inicial</td></tr> <tr><td>Po</td><td>Otra puntuación</td></tr> <tr><td>Ps</td><td>Puntuación de apertura</td></tr> <tr><td>S</td><td>Símbolo</td></tr> <tr><td>Sc</td><td>Símbolo de moneda</td></tr> <tr><td>Sk</td><td>Símbolo modificador</td></tr> </tbody> </table>	Código	Significado	Cc	Control	Cf	Formato	Cn	Sin asignar	Co	Uso privado	Cs	Sustituto	L	Letra	Li	Letra minúscula	Lm	Letra modificadora	Lo	Otra letra	Lt	Letra de título	Lu	Letra mayúscula	M	Marca	Mc	Marca de espacio	Me	Marca de cierre	Mn	Marca de no-espacio	N	Número	Nd	Número decimal	NI	Número letra	No	Otro número	P	Puntuación	Pc	Puntuación de conexión	Pd	Puntuación guión	Pe	Puntuación de cierre	Pf	Puntuación final	Pi	Puntuación inicial	Po	Otra puntuación	Ps	Puntuación de apertura	S	Símbolo	Sc	Símbolo de moneda	Sk	Símbolo modificador
Código	Significado																																																														
Cc	Control																																																														
Cf	Formato																																																														
Cn	Sin asignar																																																														
Co	Uso privado																																																														
Cs	Sustituto																																																														
L	Letra																																																														
Li	Letra minúscula																																																														
Lm	Letra modificadora																																																														
Lo	Otra letra																																																														
Lt	Letra de título																																																														
Lu	Letra mayúscula																																																														
M	Marca																																																														
Mc	Marca de espacio																																																														
Me	Marca de cierre																																																														
Mn	Marca de no-espacio																																																														
N	Número																																																														
Nd	Número decimal																																																														
NI	Número letra																																																														
No	Otro número																																																														
P	Puntuación																																																														
Pc	Puntuación de conexión																																																														
Pd	Puntuación guión																																																														
Pe	Puntuación de cierre																																																														
Pf	Puntuación final																																																														
Pi	Puntuación inicial																																																														
Po	Otra puntuación																																																														
Ps	Puntuación de apertura																																																														
S	Símbolo																																																														
Sc	Símbolo de moneda																																																														
Sk	Símbolo modificador																																																														

símbolo	significado	
	Código	Significado
	Sm	Símbolo matemático
	So	Otro símbolo
	Z	Separador
	Zl	Separador de línea
	Zp	Separador de párrafo
	Zs	Separador de espacio
i	La letra i al final de los delimitadores permite ignorar mayúsculas y minúsculas.	

funciones de expresiones regulares

función	significado
<pre>preg_match(patrón, string [, arrayResult] [, flag] [,despl])</pre>	Indica una expresión regular (apartado patrón) y la comprueba en el String que se pasa a la función. Devuelve verdadero en el caso de que el texto cumpla la expresión regular. arrayResult es un array que se puede indicar para almacenar en él el texto dentro del String que cumple el patrón. De modo que el elemento cero del array será el texto completo que cumpla todo el patrón, el elemento 1 será el texto que cumpla el primer subpatrón que se haya definido (los subpatrones se indican entre paréntesis), etc. El parámetro flags permite utilizar la constante PREG_OFFSET_CAPTURE para que el array anterior almacene además de cada texto que cumpla la condición, la posición en la que se estaba ese texto dentro del array original (lo coloca en el array detrás de cada texto que cumpla la condición) despl permite indicar el índice dentro del String por el que se empezará a buscar el patrón (si no se indica este parámetro comenzamos a buscar por el principio).
<pre>preg_match_all(patrón, string [, arrayResult] [, flag] [,despl])</pre>	Igual que la anterior, sólo que cuando encuentra el patrón, sigue buscando en el resto del texto para buscar la siguiente aparición. >Esto la hace más útil para usar arrays de resultados.

función	significado
preg_replace (patrón, reemplazo, string [, límite] [, cuenta])	Busca el patrón en el string indicado y cuando le encuentra, devuelve un nuevo string resultado de sustituir el patrón encontrado por el texto de reemplazo indicado. Ejemplo: <pre>\$s="Este es un documento Java, y no de la isla de Java"; echo preg_replace("/java/i", "PHP",\$s);</pre> Escribe <i>Este es un documento PHP, y no de la isla de PHP</i> El patrón puede incluso ser un array con varios patrones: si es así, se sustituye cada patrón del array en el string por el texto de reemplazo indicado. Es posible que incluso el reemplazo sea un array de textos de reemplazo. Si es así: cada elemento del array de patrones se reemplaza por el elemento del array de reemplazo correspondiente. El parámetro opcional <i>límite</i> pone un tope al número de reemplazos efectuados. Por ejemplo si vale 1, sólo se reemplaza la primera aparición del patrón. Por defecto vale -1 (es decir, no hay límite). La variable <i>cuenta</i>, si se utiliza, sirve para almacenar el número de reemplazos efectuados.
preg_split (patrón, string [, límite] [, flags])	Divide el string indicado según el patrón de expresión regular indicado y devuelve un array que contiene cada trozo obtenido del string. Ejemplo: <pre>\$s="texto a dividir, ole y ole"; print_r (preg_split("/[\\s,]+/", \$s));</pre> Sale: <pre>Array ([0] => texto [1] => a [2] => dividir [3] => ole [4] => y [5] => ole)</pre> El parámetro opcional <i>límite</i> indica el máximo de trozos a obtener.

(3.2.9) funciones estándar de uso con strings

funciones básicas

función	significado
<code>strlen(string)</code>	Devuelve el tamaño del String

mayúsculas y minúsculas

función	significado
<code>strtolower(texto)</code>	Convierte el texto a minúsculas
<code>strtoupper(texto)</code>	Convierte el texto a mayúsculas
<code>lcfirst(texto)</code>	Retorna un string resultado de poner en minúsculas el primer carácter del texto (suponiendo que sea una letra).
<code>ucfirst(texto)</code>	Retorna un string resultado de poner en mayúsculas el primer carácter del texto (suponiendo que sea una letra).

funciones de comparación

función	significado
<code>strcmp(texto1, texto2)</code>	Compara los dos textos (strings) y devuelve cero si son iguales, uno si el primero es mayor en orden alfabético (usando el código ASCII) y -1 si es mayor el segundo string.
<code>strccasemp(texto1, texto2)</code>	Igual que la anterior pero no tiene en cuenta las mayúsculas (sólo en inglés)
<code>strnatcmp(texto1, texto2)</code>	Igual que las anteriores, pero la comparación que hace tiene en cuenta la forma natural humana de ordenar. Así por ejemplo el texto <i>imagen2</i> sería considerado menor que <i>imagen11</i> (en orden alfabético es mayor).
<code>strcasenatcmp(texto1, texto2)</code>	Igual que la anterior pero sin considerar mayúsculas ni minúsculas.
<code>levenshtein(texto1, texto2)</code>	Devuelve un número entero conocido como distancia Levenshtein que simboliza el número de modificaciones al primer texto que nos permitirían conseguir el segundo texto. De modo que si esa distancia es pequeña, los dos textos son parecidos.
<code>metaphone(texto [, fonemas])</code>	Devuelve un código que indica la forma de pronunciar una palabra (usando el inglés), de modo que dos palabras con pronunciación similar tendrían el mismo código. El parámetro <i>fonemas</i> indica el número máximo de fonemas a tener en cuenta (menos, más palabras parecidas habrá)
<code>similar_text(texto1, texto2)</code>	Devuelve el número de caracteres parecidos entre el <i>texto1</i> y el <i>texto2</i>

función	significado
<pre> texto1, texto2 [,porcentaje]) </pre>	y el texto2. En realidad lo interesante es usar el tercer parámetro porcentaje. Este parámetro se pasa por referencia por lo que tiene que ser una variable. Dicha variable recibe un porcentaje de similitud entre ambas cadenas de texto.

funciones de búsqueda y reemplazo

función	significado
strpos (texto, textoBusq)	Devuelve la posición del segundo texto dentro del primero (empieza a contar la posición desde el número cero). Si el texto buscado no se encuentra, devuelve false. Ejemplo: <pre>\$a="Esta es la comunidad de Castilla y León"; echo strpos(\$a," Castilla");</pre> Escribiría 24, ya que Castilla empieza a aparecer en la posición 24 del String \$a.
stripos (texto, textoBusq)	Igual que la anterior pero no tiene en cuenta mayúsculas ni minúsculas. Sólo es válida con textos que no usen caracteres fuera del alfabeto inglés.
strpbrk (texto, listaCars)	listaCars es un string que contiene todos los caracteres que deseamos buscar en el texto de modo que busca cualquiera de esos caracteres dentro del texto. La función si encuentra cualquiera de esos caracteres, devuelve el resto de caracteres desde la posición del primer carácter que encuentre en el texto. Ejemplo: <pre>echo strpbrk("este año no voy a la montaña","ñyn");</pre> Escribe (puesto que encuentra primero a la eñe): <pre>ño no voy a la montaña</pre>
str_replace (textoBuscado, textoReemplazo, texto [,veces])	Localiza todas las apariciones del textoBuscado en el texto que se pasa como tercer parámetro y las cambia por el texto indicado en textoReemplazo. El cuarto parámetro (opcional), veces, se envía por referencia y almacenará el número de reemplazos realizados
str_ireplace (textoBuscado, textoReemplazo, texto [,veces])	Idéntica a la anterior, pero no distingue entre mayúsculas y minúsculas.

función	significado
substr_replace (texto, textoReemplazo, posInicial [,posFinal])	Coloca en el texto original, el texto indicado como textoReemplazo , de modo que sustituya a todos los caracteres desde la posición indicada como posInicial , hasta el final o hasta el número indicado con el parámetro posFinal .
strtr (texto,arrayTraducción)	Hace reemplazos múltiples de caracteres en el texto y devuelve el texto resultante de realizar esos reemplazos. El array que se usa como segundo parámetro, contiene datos de forma que cada índice se buscará en el texto y se reemplazará por su valor. Ejemplo: <pre>\$array=array("a"=>"á","e"=>"é","i"=>"í","o"=>"ó"); echo strtr("este texto tiene", \$array)</pre> Escribe: éste téxtó tiéné
stripslash (texto)	Elimina en el texto todos los caracteres backslash (\). Es muy útil para texto procedente de lenguajes de programación.
strip_tags (texto [,noRetirables])	Retira del texto todas las etiquetas de tipo HTML o PHP que haya contenidas, excepto las que se indiquen en el parámetro noRetirables , que es un string que contendrá las etiquetas que no queremos retirar seguidas. Ejemplo: <pre>strip_tags(texto,"<p>")</pre> Elimina del texto todas las etiquetas que haya excepto p y strong .

funciones de extracción de texto

función	significado
strstr (texto, textoBusq)	Busca un string dentro de otro y devuelve los caracteres desde su primera aparición hasta el final. En la respuesta incluye el texto buscado. Ejemplo: <pre>\$a="Esta es la comunidad de Castilla y León"; echo strstr(\$a," Castilla");</pre> Escribiría Castilla y León
stristr (texto, textoBusq)	Igual que la anterior pero no tiene en cuenta mayúsculas ni minúsculas. Sólo es válida con textos que no usen caracteres fuera del alfabeto inglés.

función	significado
strtok(string [,token])	Permite extraer un texto en base a una serie de caracteres de modo que primer coge el primer texto dentro del string original (primer token) y en las siguientes llamadas irá devolviendo el resto de tokens (en las siguientes llamadas no se usa el primer parámetro) hasta que tras devolver el último retorna el valor falso. Ejemplo: <pre>\$s="este es el texto, le vamos a partir"; \$tok=strtok(\$s," "); while(\$tok!=false){ echo \$tok."\n"; \$tok=strtok(" "); }</pre> Escribe: <pre>este es el texto le vamos a partir</pre>
explode(delimitador, texto [,límite])	Devuelve un array donde cada elemento del mismo es una subcadena que procede de separar el texto indicado en base a su delimitador. El parámetro límite indica el máximo número de cadenas a extraer. Ejemplo: <pre>\$s="soy una frase, con unas, cuantas, comas"; \$a=explode(",",\$s); print_r(\$a);</pre> Escribirá: <pre>Array((0)=>soy una frase [1]=>con unas [2]=>cuantas [3]=>comas)</pre>

función	significado
implode(array)	Une todos los elementos del array para formar un texto que contiene el valor de cada elemento.
str_repeat(texto, veces)	Devuelve un string que contiene el texto indicado repetido las veces que se indiquen en el segundo parámetro.
str_shuffle(texto)	Devuelve una copia del texto donde cada carácter se ha movido aleatoriamente. Es decir, forma un anagrama.
strrev(texto)	Devuelve un string que contiene el texto original al revés.
str_split(texto [,tamaño])	Devuelve un array en el que en cada elemento hay un trozo del texto original. El texto se trocea por tamaño de caracteres; si no se indica tamaño se divide carácter a carácter; si, por ejemplo, tamaño vale 3, se divide de tres en tres caracteres.
str_word_count(texto [,formato [,listaCaracteres]])	Sin indicar segundo ni tercer parámetro, devuelve el número de palabras encontradas en el texto. El parámetro formato puede tomar los valores: 1 La función devuelve el número de palabras encontradas 2 Devuelve un array escalar con todas las palabras encontradas 3 Devuelve un array asociativo donde cada valor es cada palabra encontrada y su índice la posición en el texto original. listaCaracteres es un string que contiene caracteres que en esta función se deben de considerar como parte de la palabra y no como separadores de caracteres. Es muy útil para texto escrito en cualquier lengua distinta del inglés y así considerar a la ñe como parte normal del texto.

funciones para extraer subcadenas de texto

función	significado
substr(texto, posInicial [,tamaño])	Extrae del string que se pasa como primer parámetro, el texto que va desde la posición indicada (empezando a contar por cero) hasta el final. O bien, extrae desde dicha posición el número de caracteres indicados por el parámetro tamaño .

funciones de limpieza de texto

función	significado
<code>trim(texto, [caracteres])</code>	Sin usar el segundo parámetro, elimina del texto los espacios en blanco del principio y el final. Elimina también saltos de línea, retornos de carro, tabuladores, nulos y caracteres de salto vertical. El segundo parámetro permite indicar una lista de caracteres entrecomillada que serán los que se eliminen si se encuentran al final y al principio del texto (en lugar de eliminar los espacios).
<code>ltrim(texto, [caracteres])</code>	Igual que el anterior, pero sólo elimina los caracteres al inicio del texto.
<code>rtrim(texto, [caracteres])</code>	Igual que el anterior, pero sólo elimina los caracteres al final del texto.

obtener códigos ASCII

función	significado
<code>chr(códigoASCII)</code>	Devuelve el carácter correspondiente al código ASCII indicado.
<code>ord(carácter)</code>	Inversa a la anterior. Devuelve el código ASCII del carácter indicado.

funciones de formato de datos

función	significado
<code>number_format(número [,decimales [,caracterDecimal [,caracterDeMiles]])</code>	Devuelve un string que contiene al número que se indica formateado de modo que aparece el separador de miles. El segundo parámetro (opcional) permite indicar cuántos decimales vamos a utilizar para mostrar el número. Los otros dos parámetros se utilizan para indicar cuál es el carácter de separador de decimales y el de miles. Ejemplo: <pre>echo number_format(1234567.892,2,"",".");</pre> Muestra: 1.234.567,89

función	significado
money_format (formato, número)	Devuelve un string resultante de aplicar al número que se recibe como segundo parámetro el formato de moneda indicado mediante el primer parámetro. Sólo funciona en entornos compatibles con la función strfmon de lenguaje C (Windows no lo es). El parámetro formato es, a su vez, un string que contiene símbolos especiales para dar formato. Esa expresión está precedida por el símbolo % al que le pueden seguir uno o más de estos caracteres i Formatea el número usando los símbolos y separadores pertinentes según lo especificado por la función setlocale basándose en el sistema monetario internacional. Por ejemplo si antes hemos usado setlocale("LC_ALL", "es-ES"), el formato del número será el correspondiente al formato monetario español. n Idéntica al anterior pero usa el sistema nacional de moneda, según la configuración regional especificada por setlocale. =f Mediante el símbolo =, se especifica un carácter de relleno para el número (por defecto se usa el espacio en blanco). ^ Deshabilita las opciones de agrupamiento (el separador de miles) + Indica signo para el número (Muestra los números negativos entre paréntesis ! Elimina el símbolo de moneda #n Caracteres de anchura que se usarán para la parte entera del número. .p Número de decimales que se utilizarán - Alineación izquierda del número (normalmente la alineación es derecha)

cifrado

función	significado
md5 (texto [,raw_input])	Devuelve hash correspondiente al texto indicado usando el algoritmo MD5. Si el parámetro raw_input se indica con valor true (por defecto vale false), codifica en binario crudo con tamaño 16.
sha1 (texto [,raw_input])	Igual que el anterior pero utilizando el algoritmo SHA1

función	significado
<code>hash(</code> algoritmo, texto [,raw_input] <code>)</code>	Genera el hash correspondiente a aplicar sobre el texto el algoritmo que se indique en el parámetro <i>algoritmo</i> (puede ser “sha256” , “md5”). La lista completa de posibles algoritmos se puede consultar con la función <i>hash_algos</i> que devuelve un array con todos los nombres posibles a utilizar.
<code>crc32(texto)</code>	Genera el polinomio CRC32 de comprobación sobre el string indicado. Mediante ese polinomio podemos comprobar la validez de los datos.
<code>crypt(texto, salt)</code>	Función completa para cifrar el texto indicado. Su funcionamiento es complejo.
<code>str_rot13(texto)</code>	Codifica el texto utilizando la rotación ROT13 de cifrado.

(3.3) funciones de fecha

Como en todos los lenguajes el manejo de fechas y horas es complejo. Pero PHP proporciona numerosas funciones que facilitan su manejo.

PHP utiliza el formato de fecha y hora de Unix, por lo que muchas funciones requieren pasar las fechas en este formato. Por ello hay otras muchas funciones que nos ayudan a crear fechas en ese formato a partir de un texto que represente a una fecha.

función	significado
<code>time()</code>	Devuelve la fecha y hora actual en el formato nativo (Unix) de PHP.
<code>strftime(formato[,fecha])</code>	Devuelve un texto que representa la fecha actual (o la que se indique como segundo parámetro) en el formato regional establecido con setlocale . El formato puede incluir estos símbolos: %a Día de la semana (tres letras) %A Día de la semana (completo) %d El día del mes con dos dígitos (del 01 al 31) %e El día del mes (de 1 a 31) %j Día del año, 3 dígitos (del 001 al 366) %u Día de la semana (del 1 al 7) %w Día de la semana (del 0 al 6) %W Número de semana del año, comenzando con el primer Domingo como la primera semana

función	significado
	%b Nombre del mes con tres letras %B Nombre del mes completo %m Número de mes en dos cifras (del 01 al 12) %C Número de siglo (por ejemplo 21) %y Año en dos cifras %Y Año en cuatro cifras %H Hora en formato 24 horas (del 00 al 23) %k Hora en formato 24 horas (del 0 al 23) %I Hora en formato de 12 horas (01 al 12) %l La hora en formato de 12 horas (del 1 al 12) %M Minutos en dos cifras (del 01 al 59) %p 'AM' o 'PM' en MAYÚSCULAS basados en la hora dada %P 'am' o 'pm' en minúsculas basados en la hora dada %S Segundos en dos dígitos (del 00 al 59) %X Representación preferida de la hora basada en la configuración regional, sin la fecha
date (formato [,fecha])	Da formato a la fecha. Si no se indica segundo parámetro, se toma la fecha actual. El formato puede llevar estos símbolos (se indican los más importantes): d Día del mes (del 01 al 31) D Día de la semana con tres letras (en inglés) j Día del mes (del 1 al 31) l Día de la semana (en inglés) w Día de la semana (del 1 al 7) z Día del año (del 1 al 365 ó 366) F Nombre del mes (en inglés) m Mes (del 01 al 12) M Mes con tres letras (en inglés) n Mes (del 1 al 12) t Número de días del mes dado (28,29,30 ó 31)

función	significado
	L Verdadero si el año es bisiesto y Año con dos cifras Y Año con cuatro cifras a Símbolo AM en minúsculas A AM en mayúsculas h Hora en formato 12 horas (del 01 al 12) H Hora en formato 24 horas (del 00 al 23) g Hora en formato 12 horas (del 1 al 12) G Hora en formato 24 horas (del 0 al 23) i Minutos (del 00 al 59) s Segundos (del 00 al 59) u Microsegundos
mktime([hora [,minuto [,segundo [,mes [,día [,año [época]]]]]]])	Crea una fecha y hora a partir de los parámetros indicados. Los que no se indiquen se toman de la fecha y hora actuales. El último parámetro indica con un 1 que estamos en horario de verano y con un -1 en el de invierno.
strtotime(fecha, formato)	Devuelve un array asociativo que contiene como valores los datos de la fecha indicada. El formato cumple los posibles valores del parámetro formato de la función strtotime explicada anteriormente.
local_time([fecha [,asociativo]])	Devuelve la fecha actual (o la que se indique como primer parámetro) en forma de array en el que cada elemento contiene cada parte de la fecha y hora. Se puede indicar una fecha concreta y un valor verdadero como segundo parámetro para indicar que deseamos un

función	significado
	array asociativo en lugar de escalar.
strtotime(texto)	Analiza el texto y devuelve la fecha correspondiente. Ejemplos de uso²: <pre><?php echo strtotime("now"), "\n"; echo strtotime("10 September 2000"), "\n"; echo strtotime("+1 day"), "\n"; echo strtotime("+1 week"), "\n"; echo strtotime("+1 week 2 days 4 hours 2 seconds"), "\n"; echo strtotime("next Thursday"), "\n"; echo strtotime("last Monday"), "\n"; ?></pre>
checkdate(mes, día, año)	Devuelve verdadero si la fecha indicada de esta forma es válida.
date_default_timezone_get()	Devuelve la zona horaria en uso (por ejemplo Europe/Berlin)
date_default_timezone_set(zona)	Establece la nueva zona horaria. EL texto debe ser estándar, por ejemplo (Europe/Berlin).

² Ejemplo #1 de la documentación de php.net